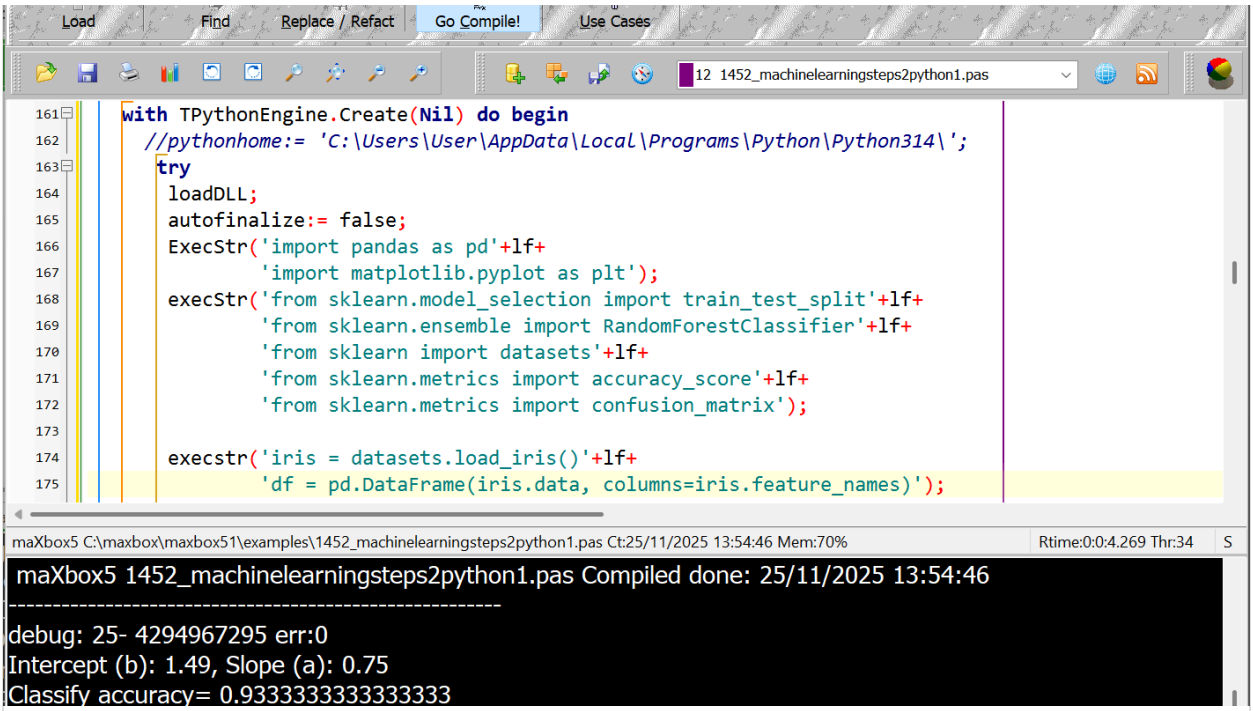


Machinelearning Steps

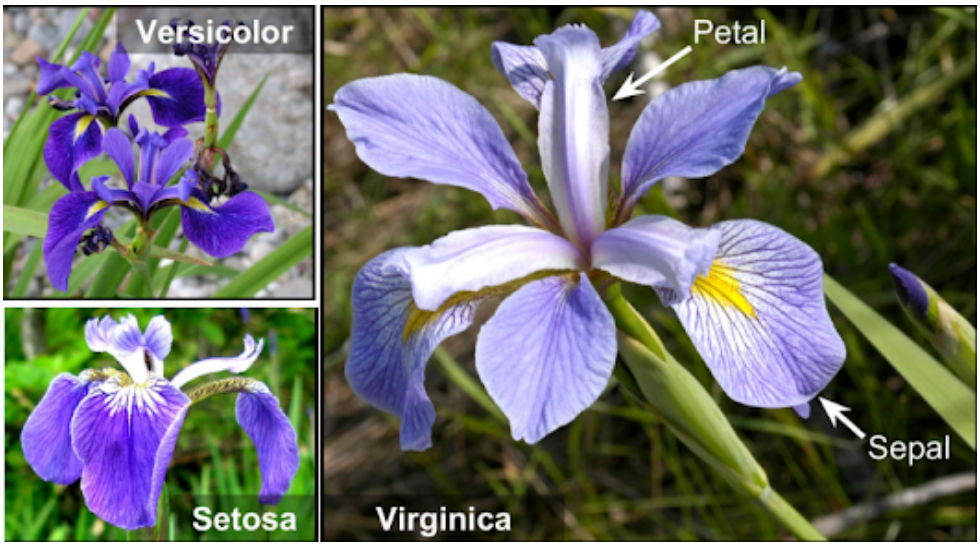
November 25, 2025

This article will provide the clear cut understanding of Iris dataset and how to do classification (supervised learning) on Iris flowers dataset using python, seaborn and sklearn.

The Iris dataset is often used in machine learning and data science courses, because it's simple to understand and well-defined, yet interesting enough to present real challenges to new learners. This single code snippet will use Python with Python4Delphi to classify the Iris dataset into one of three flower species: Setosa, Versicolor, or Virginica.

Download 1452_machinelearningsteps2python12plotdataset.pas (maXbox5)

The iris data consisted of 150 samples of three species of Iris. The first column represented sepal length, the second column represented sepal width, the third column represented petal length, and the fourth column represented petal width. I'm going to use sci-kit-learn to classify these instances according to their species.



Advertisement



Bathroom Upgrade Made Easy

Transform your shower with a double-rod curtain setup for a custom, designer look without renovation.

See the full guide

Privacy Settings

The provided Delphi code snippet uses the Python4Delphi library, specifically the TPythonEngine component, to embed Python code execution into a Delphi application. It creates an instance of TPythonEngine and uses it to run a sequence of Python commands that implement a machine learning workflow with the iris dataset using popular Python libraries like pandas, matplotlib, pyplot, scikit-learn (sklearn), and seaborn.



200 Python Machine Learning in Delphi maXbox5

Max Kleiner





Watch on

explainer video, 5:45

Import Libraries

Advertisement
Privacy Settings

First we load the libraries:

Before diving in, ensure as prerequisites you have these libraries installed:

```
pip install pandas numpy matplotlib seaborn scipy scikit-learn stat
```

It executes multiple Python import statements to load common data science and machine learning libraries:

pandas (for data manipulation), matplotlib.pyplot (plotting),
sklearn's modules including train_test_split,
RandomForestClassifier, for supervised learning
dataset loaders, and metrics like accuracy_score and confusion_matrix
and It loads the iris dataset
from sklearn.datasets and converts it into a pandas DataFrame.

```
with TPythonEngine.Create(Nil) do begin
     //pythonhome:= 'C:\Users\User\AppData\Local\Programs\Python\P
  try
   loadDLL;
   autofinalize:= false;
   ExecStr('import pandas as pd'+lf+
        'import matplotlib.pyplot as plt');
   execStr('from sklearn.model_selection import train_test_split'+
        'from sklearn.ensemble import RandomForestClassifier'+lf+
        'from sklearn import datasets'+lf+
        'from sklearn.metrics import accuracy_score'+lf+
        'from sklearn.metrics import confusion_matrix');
```

Load Dataset

But the data are not well structured for us to understand. So we need to convert it into a pandas DataFrame. Pandas is a great tool for doing all sorts of things related to datasets, including preprocessing and exploring them.

```
execstr('iris = datasets.load_iris()'+lf+
        'df = pd.DataFrame(iris.data, columns=iris.feature_names)')
```

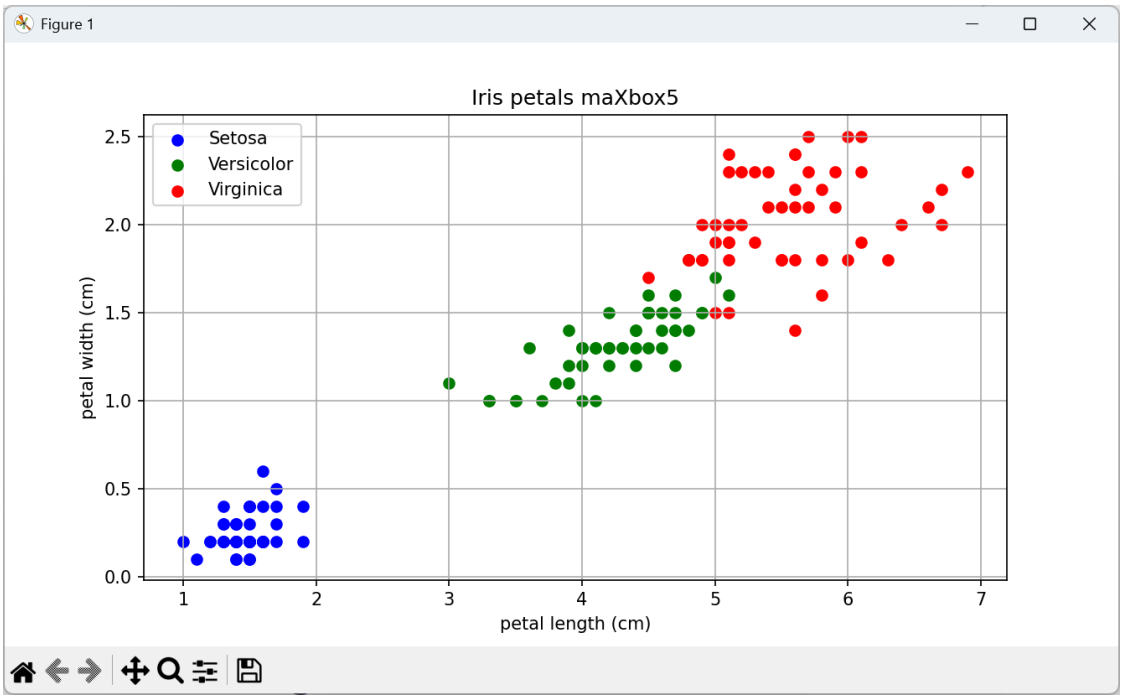
- It loads the iris dataset from sklearn.datasets and converts it into a pandas DataFrame.
- The data is split into training and test sets with stratification and fixed random seed as random_state for reproducibility and comparision.

Advertisement
Privacy Settings

Explore Dataset

The dataset contains **150 samples** of iris flowers, divided into three species: **Setosa**, **Versicolor**, and **Virginica**. Each sample is described by **four features** measured in centimeters:

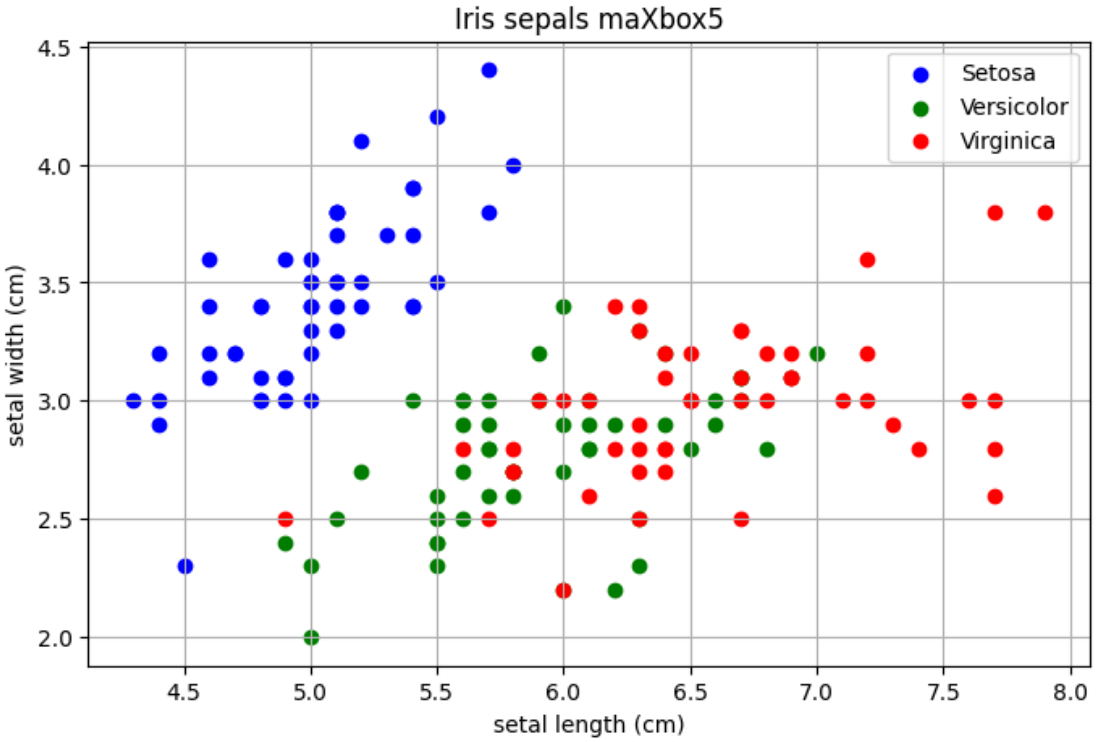
- **Sepal Length**: The length of the flower's sepal.
- **Sepal Width**: The width of the flower's sepal.
- **Petal Length**: The length of the flower's petal.
- **Petal Width**: The width of the flower's petal.



'# lables and scatter points fig, ax = plt.subplots()

Advertisement

Privacy Settings



Iris Sepals features

Code for explore

GroupBy is not just about aggregation. (GroupBy Is Underrated)
It lets you compute group-wise features,
normalize values, build windowlike metrics, combine multiple views of your data and s
tructure complex datas.

```
irisd = datasets.load_iris()
irisdf = pd.DataFrame(
        data= np.c_[irisd['data'],irisd['target']],
        columns= irisd['feature_names']+['target']
        )
species = []
for i in range(len(irisd['target'])):
   if irisd['target'][i] == 0:
     species.append('setosa')
   elif irisd['target'][i] == 1:
     species.append('versicolor')
   else:
     species.append('virginica')
irisdf['species'] = species

"""Now we can print out a scatter plot:"""

print(irisdf.groupby('species').size())
setosa = irisdf[irisdf.species =='setosa']
versicolor = irisdf[irisdf.species =='versicolor']
virginica = irisdf[irisdf.species =="virginica"]
fig, ax = plt.subplots()
fig.set_size_inches(10, 5) # adjusting the length and width of plo
# lables and scatter points
ax.scatter(setosa["petal length (cm)"],setosa["petal width (cm)"],
ax.scatter(versicolor["petal length (cm)"],versicolor["petal width
```

```
ax.scatter(virginica["petal length (cm)"],virginica["petal width (
ax.set_xlabel("petal length (cm)")
ax.set_ylabel("petal width (cm)")
ax.grid()
ax.set_title("Iris petals maXbox5")
ax.legend()
plt.show()
```

Classify the species

So we don't need to hardcode the algorithm for classification since there are already many algorithms available in the sci-kit learn package. We can simply choose any of them and use them. Here, I am going to use the Random Forest Classifier model. Now, after training our model on training data, we can predict petal and sepal measurements on testing data as the features. And that's the non binary cassification for the first block!

```
execstr('X_train,X_test, y_train,y_test = train_test_split(df[iris.
        'iris.target,test_size=0.5, stratify=iris.target, random_st
execstr('rf = RandomForestClassifier(n_estimators=100, oob_score=Tr
                                     'random_state=123456)'+lf+'rf.

writeln('Classify accuracy= '+Evalstr('accuracy_score(y_test, rf.pr
```

Also, we can use the train_test_split function in sklearn in order to split the dataset into train and test.

Advertisement



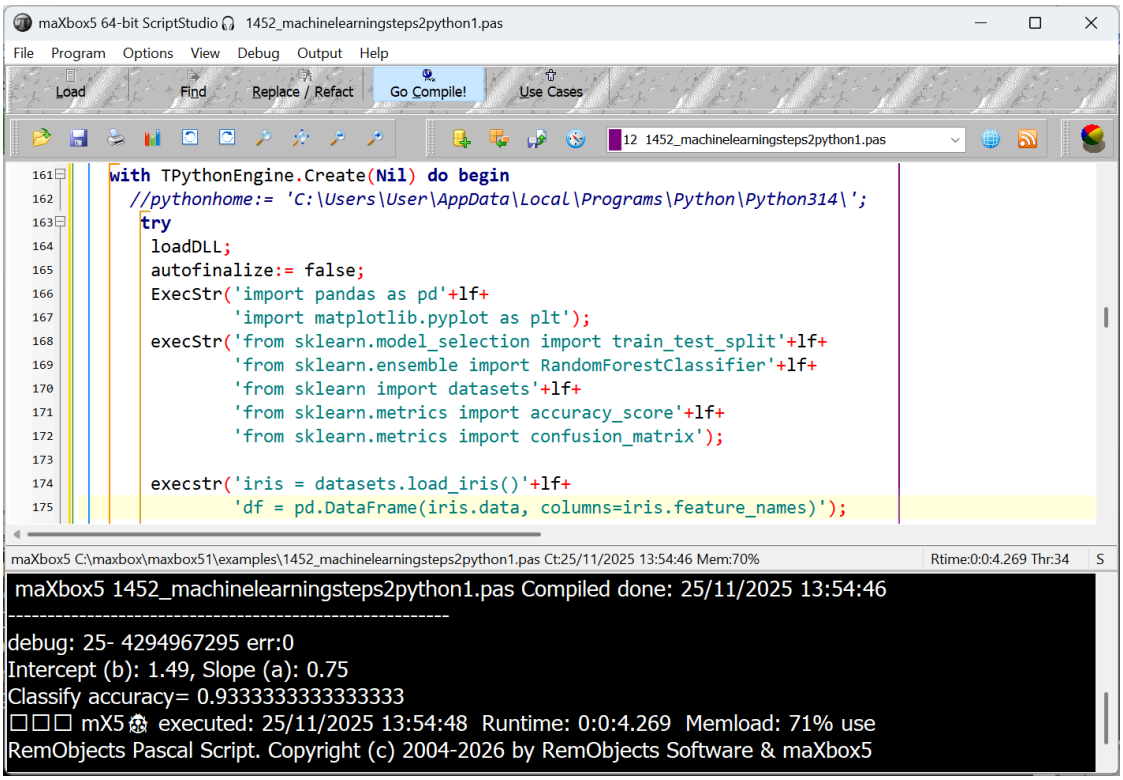
Punks Took the Crown?

Pete Townshend's surprising take: punk wasn't just rebellion—it was rock's reluctant heir apparent.

Learn more here

Privacy Settings

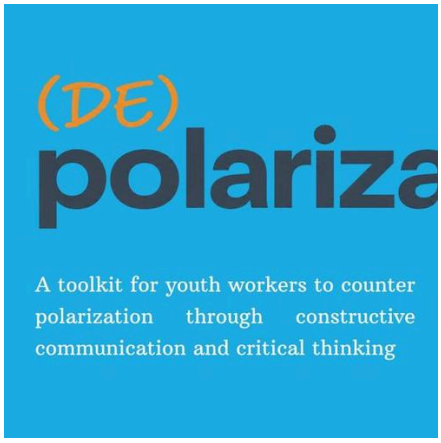
We get Classify accuracy= 0.9333333333333333



https://sourceforge.net/projects/maxbox5/files/examples/1452_machinelearningsteps2python1.pas/download

Plotting the Performance Measures

Advertisement



Navigating Group Divides

Is polarization disrupting your youth group? Learn how to keep conversations open and connections strong.

Explore the Workbook

Privacy Settings

Performance measures are used to evaluate effectiveness of classifiers on different datasets with different characteristics. For classification problems, there are three main measures for evaluating the model, the precision (the accuracy of positive predictions or the number of most relevant values from retrieved values.), Recall (ratio of positive instances that are truly detected by the classifier), and confusion matrix.

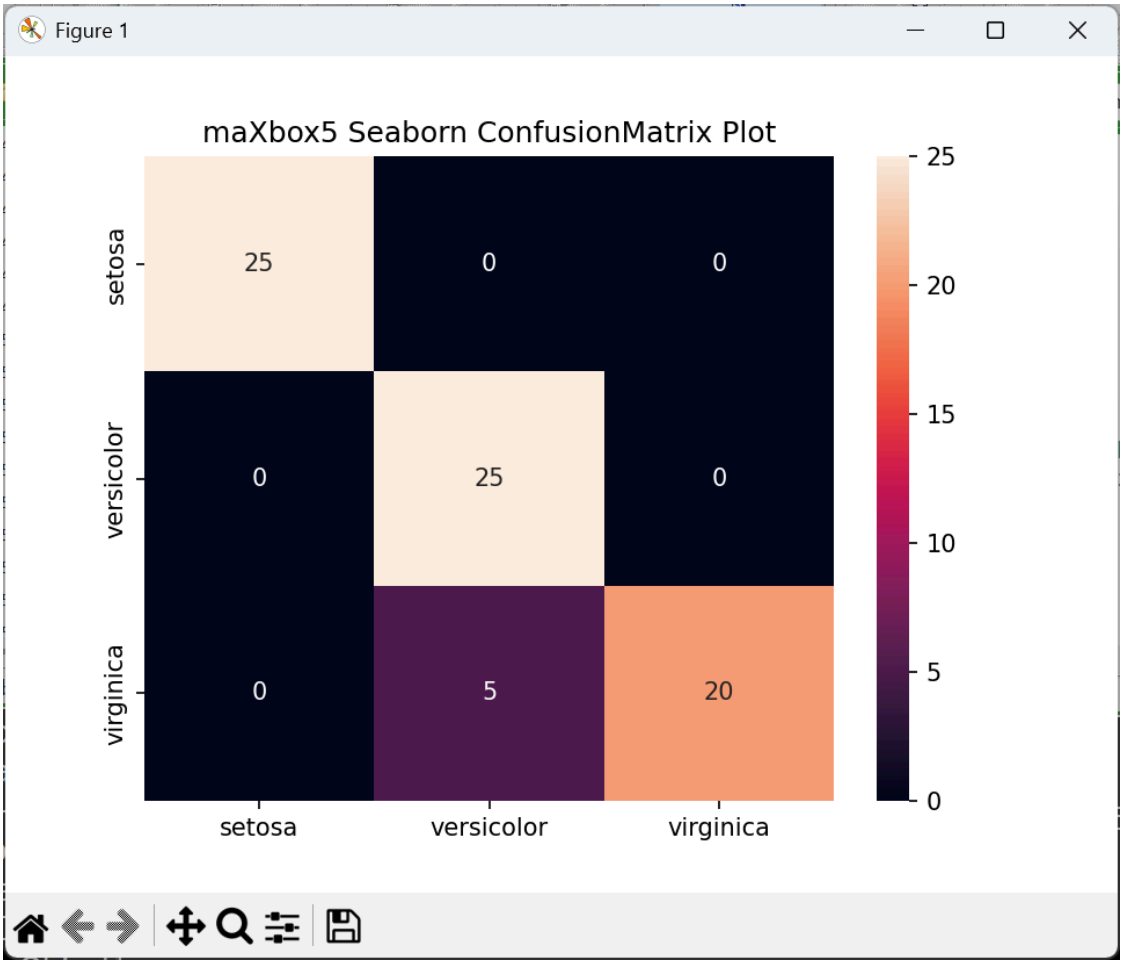
In other words if you don't recognize a person then its bad recall and if you recognize a person for another (confuse with) then its bad precision!

Mit anderen Worten: Wenn man eine Person nicht erkennt, dann ist das ein schlechtes Erinnerungsvermögen, und wenn man eine Person mit einer anderen verwechselt,

dann ist das eine schlechte Genauigkeit!

En d'autres termes, si vous ne reconnaissez pas une personne, c'est que votre rappel est mauvais, et si vous reconnaissez une personne pour une autre (confusion), c'est que votre précision est mauvaise !

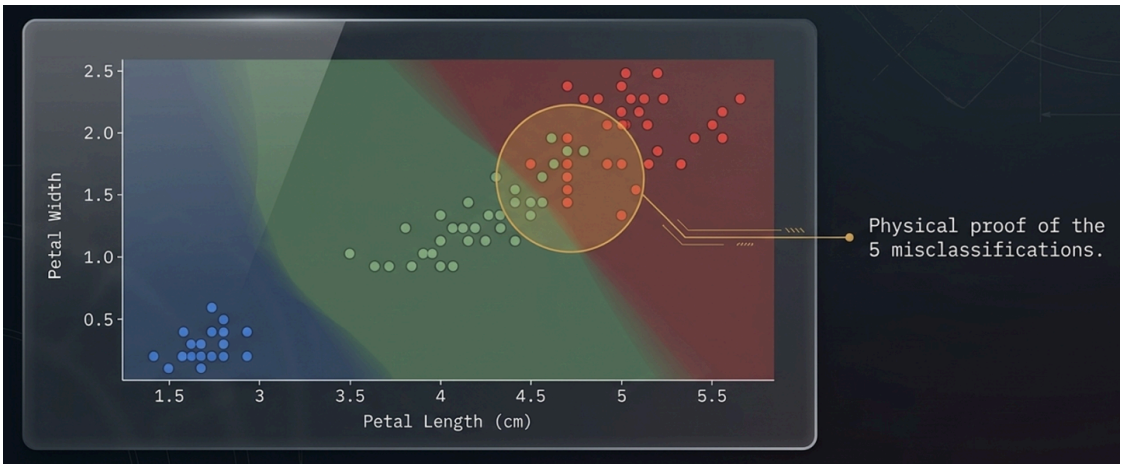
言い換えると、ある人物を認識できない場合はリコールが悪いですし、ある人物を別の人物と認識した場合（混同した場合）、精度が悪いということです。



Confusion Matrix for Performance Measure

Advertisement
Privacy Settings

The scores are pretty good in this case only 5 misclassified. The main usage of the confusion matrix is to identify how many of the classes are misclassified by the classifier (false-positive or false-negative).



Vwesicolor has 5 False Positivs

With the following call you get a report of the performance:

execstr('print(metrics.classification_report(y_test, rf.predict(X_test), digits=3))');

precision recall f1-score support eg: Setosa(0), Versicolor(1), Virginica(2)

```
                precision recall    f1-score     support
          0     1.000     1.000     1.000        25
          1     0.833     1.000     0.909        25
          2     1.000     0.800     0.889        25

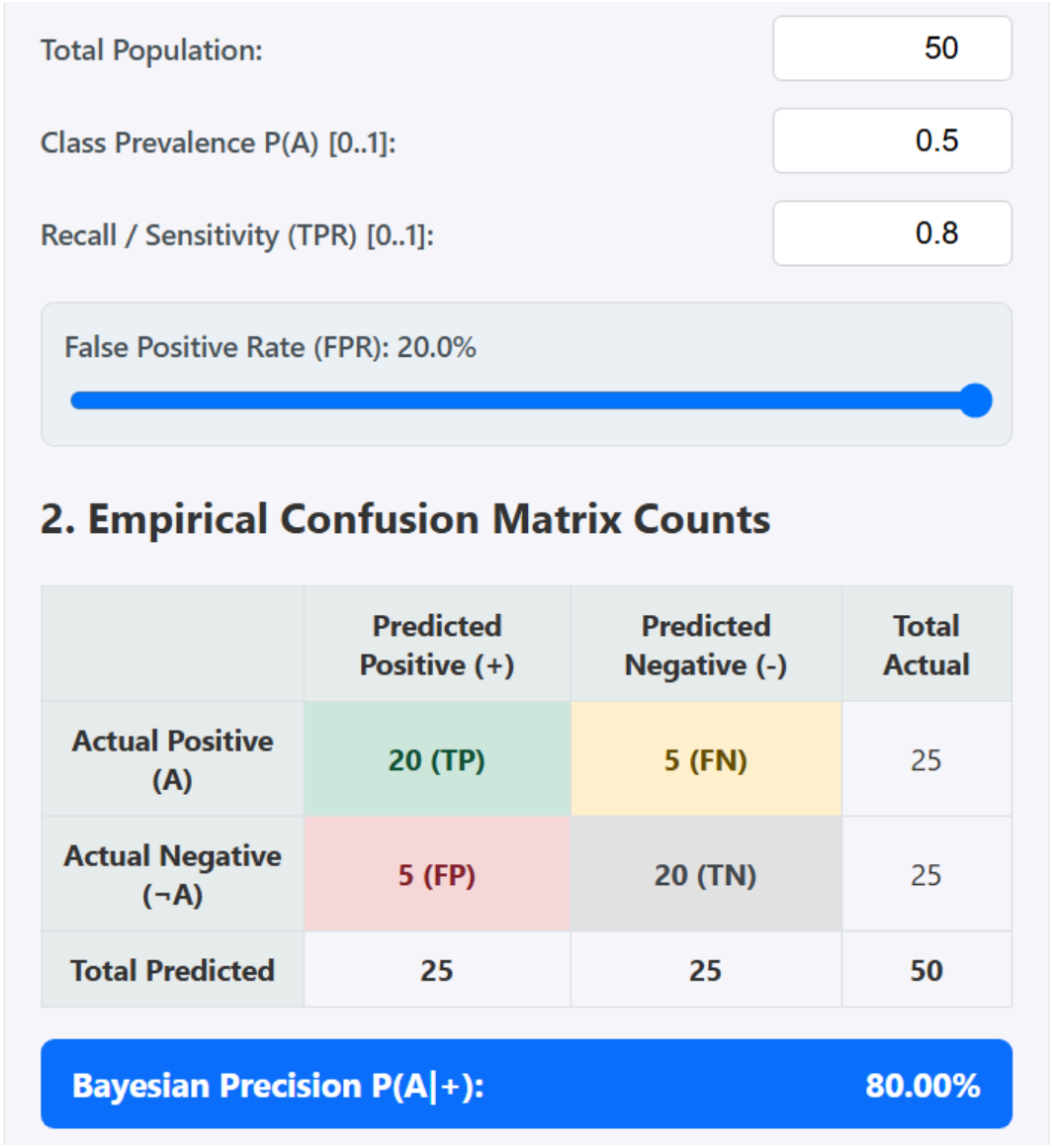
accuracy                            0.933        75
```

macro avg 0.944 0.933 0.933 75
weighted avg 0.944 0.933 0.933 75

Advertisement
Privacy Settings

We check the **recall** of virginica (2) its **80%**, that means only 80% are recognized **(recall)** as virginica so **20%** are misclassified as versicolor means 20/25= 0.8 recall!!

On the other side the **precision** of versicolor (1) is only **83%** because 5 from virginica are misclassified as versicolor and that means 25/30= 0.8333 precision!



Bayes' Theorem to Confusion Matrix & Precision-FPR Plotter of maXbox5

Advertisement
Privacy Settings

Total Population: 50

Class Prevalence P(A) [0..1]: 0.5

Recall / Sensitivity (TPR) [0..1]: 1

False Positive Rate (FPR): 20.0%

2. Empirical Confusion Matrix Counts

	Predicted Positive (+)	Predicted Negative (-)	Total Actual
Actual Positive (A)	25 (TP)	0 (FN)	25
Actual Negative (¬A)	5 (FP)	20 (TN)	25
Total Predicted	30	20	50

Bayesian Precision P(A|+): 83.33%

Total Population: 50

Class Prevalence P(A) [0..1]: 0.5

Recall / Sensitivity (TPR) [0..1]: 0.8

False Positive Rate (FPR): 20.0%

2. Empirical Confusion Matrix Counts

	Predicted Positive (+)	Predicted Negative (-)	Total Actual
Actual Positive (A)	20 (TP)	5 (FN)	25
Actual Negative (¬A)	5 (FP)	20 (TN)	25
Total Predicted	25	25	50

Bayesian Precision P(A|+): 80.00%

Total Population: 50

Class Prevalence P(A) [0..1]: 0.5

Recall / Sensitivity (TPR) [0..1]: 1

False Positive Rate (FPR): 20.0%

2. Empirical Confusion Matrix Counts

	Predicted Positive (+)	Predicted Negative (-)	Total Actual
Actual Positive (A)	25 (TP)	0 (FN)	25
Actual Negative (¬A)	5 (FP)	20 (TN)	25
Total Predicted	30	20	50

Bayesian Precision P(A|+): 83.33%

Recall & Precision

Advertisement
Privacy Settings

Conclusion and Summary

The provided Delphi code snippet uses the Python4Delphi library, specifically the TPythonEngine component, to embed Python code execution into a Delphi application. It creates an instance of TPythonEngine and uses it to run a sequence of Python commands that implement a machine learning workflow with the iris dataset using popular Python libraries like pandas, matplotlib, scikit-learn, and seaborn.

Here are the detailed steps in 9 points and purpose of the code for your tests and document:

- It initializes the Python engine component (TPythonEngine.Create(nil)) and loads the Python DLL dynamically (loadDLL) as 3.13 or 3.14.
- The code sets autofinalize to false to control Python finalization manually.
- It executes multiple Python import statements to load common data science and machine learning libraries: pandas (for data manipulation), matplotlib.pyplot (plotting), sklearn's modules including train_test_split, RandomForestClassifier, dataset loaders, and metrics like accuracy_score and confusion_matrix and It loads the iris dataset from sklearn.datasets and converts it into a pandas DataFrame.
- The data is split into training and test sets with stratification and fixed random seed for reproducibility.
- A RandomForestClassifier model is created and trained on the training data.
- The accuracy of the model on the test set is evaluated and printed using the Python accuracy_score function and a confusion matrix is generated, converted into a pandas DataFrame, and plotted using seaborn's heatmap with an annotated matrix plot.
- The plot is finalized with a title and displayed using matplotlib.
- Exception handling surrounds the block to raise any Python-related errors caught during execution.
- Finally, resources are freed explicitly by calling Free on the TPythonEngine instance.

The following code showcases an integration approach where Delphi acts as a container to drive Python-based data science workflows, combining the strengths of Delphi in application development and Python in machine learning and visualization. The use of the embedded Python environment allows direct execution of Python commands from Delphi, enabling rich interaction between the two languages.

```
procedure PY_Machinelearning_steps;
 begin
   with TPythonEngine.Create(Nil) do begin
     //pythonhome:= 'C:\Users\User\AppData\Local\Programs\Python\P
    try
     loadDLL;
     autofinalize:= false;
     ExecStr('import pandas as pd'+lf+
```

```
              'import matplotlib.pyplot as plt');
      execStr('from sklearn.model_selection import train_test_split
              'from sklearn.ensemble import RandomForestClassifier'
              'from sklearn import datasets'+lf+
              'from sklearn.metrics import accuracy_score'+lf+
              'from sklearn.metrics import confusion_matrix');

      execstr('iris = datasets.load_iris()'+lf+
              'df = pd.DataFrame(iris.data, columns=iris.feature_na
      execstr('X_train,X_test, y_train,y_test= train_test_split(df[
              'iris.target,test_size=0.5, stratify=iris.target, ran
      execstr('rf = RandomForestClassifier(n_estimators=100, oob_sc
                                           'random_state=123456)'+
              'rf.fit(X_train, y_train)');

      writeln('Classify accuracy= '+evalstr('accuracy_score(y_test,

  //https://www.blopig.com/blog/2017/07/using-random-forests-in-pyth
      execstr('import seaborn as sns');
      execstr('cm = pd.DataFrame(confusion_matrix(y_test, rf.predic
                      'columns=iris.target_names, index=iris.target
      execstr('sns.heatmap(cm, annot=True)'+lf+
              'plt.title("maXbox5 Seaborn ConfusionMatrix Plot")'+l
              'plt.show()');
      except
        raiseError;
      finally
        Free;
      end;
    end;
  end;
```

References:

Advertisement
Privacy Settings

- The code sequence corresponds to classical iris dataset classification using Random Forest, a well-known machine learning example.
- maXbox5 as the scripting-studio compiler to execute code.

This code snippet can form a comprehensive description for your document highlighting how Delphi and Python4Delphi facilitate machine learning integration through Python scripts executed inside Delphi or maXbox applications. So with maXbox you we can think out of the box.

Support Vector Machine

```
procedure PY_Machinelearning_steps_SVM;
begin
  with TPythonEngine.Create(Nil) do begin
    // pythonhome := 'C:\Users\User\AppData\Local\Programs\Python\
    try
      loadDLL;
      autofinalize:= false;
```

```
      // 1. Import libraries (using SVC instead of RandomForestCla
      ExecStr('import pandas as pd' + lf + 'import matplotlib.pypl
      execStr('from sklearn.model_selection import train_test_spli
              'from sklearn.svm import SVC' + lf +
              'from sklearn import datasets' + lf +
              'from sklearn.metrics import accuracy_score' + lf +
              'from sklearn.metrics import confusion_matrix');

      // 2. Load Iris dataset into pandas DataFrame
      execstr('iris = datasets.load_iris()' + lf +
              'df = pd.DataFrame(iris.data, columns=iris.feature_n

      // 3. Perform 50/50 train/test stratified split
      execstr('X_train,X_test, y_train,y_test = train_test_split(d
              'iris.target,test_size=0.5, stratify=iris.target, ra

      // 4. Train Support Vector Classifier (RBF Kernel)
      execstr('svm = SVC(kernel=''rbf'', C=1.0, random_state=12345
              'svm.fit(X_train, y_train)');

      // 5. Output prediction accuracy
      writeln('Classify accuracy= '+ evalstr('accuracy_score(y_tes

      // 6. Generate and render Seaborn confusion matrix plot
      execstr('import seaborn as sns');
      execstr('cm = pd.DataFrame(confusion_matrix(y_test, svm.pred
              'columns=iris.target_names, index=iris.target_names)
      execstr('sns.heatmap(cm, annot=True, cmap=''Blues'')' + lf +
              'plt.title("maXbox5 Seaborn ConfusionMatrix Plot (SV
              'plt.xlabel("Predicted")' + lf +
              'plt.ylabel("Actual")' + lf +
              'plt.show()');
    except
      raiseError;
    finally
      Free;
    end;
  end;
end;
```

It adapts the Python4Delphi machine learning pipeline from your source to use a Support Vector Machine (SVM) classifier (`sklearn.svm.SVC`) instead of a Random Forest.

Support Vector Machine 2

```
procedure PY_Machinelearning_steps_SVM_2;
begin
  with TPythonEngine.Create(Nil) do begin
    //pythonhome := 'C:\Users\User\AppData\Local\Programs\Python\P
    try
      loadDLL;
      autofinalize:= false;
      // 1. Import libraries (using SVC instead of RandomForestCla
      ExecStr('import pandas as pd' + lf + 'import matplotlib.pypl
      execStr('from sklearn.model_selection import train_test_spli
              'from sklearn.svm import SVC' + lf +
              'from sklearn import datasets' + lf +
              'from sklearn.metrics import accuracy_score' + lf +
```

```
                'from sklearn.metrics import classification_report'
                'from sklearn.metrics import confusion_matrix');
        // 2. Load Iris dataset into pandas DataFrame
        execstr('import sys,io; output = io.StringIO(); sys.stdout =
        execstr('iris = datasets.load_iris()' + lf +
                'df = pd.DataFrame(iris.data, columns=iris.feature_n

        // 3. Perform 50/50 train/test stratified split
        execstr('X_train,X_test, y_train,y_test = train_test_split(d
                'iris.target,test_size=0.5, stratify=iris.target, ra

        // 4. Train Support Vector Classifier (RBF Kernel)
        execstr('svm = SVC(kernel=''rbf'', C=1.0, random_state=12345
                'svm.fit(X_train, y_train)');

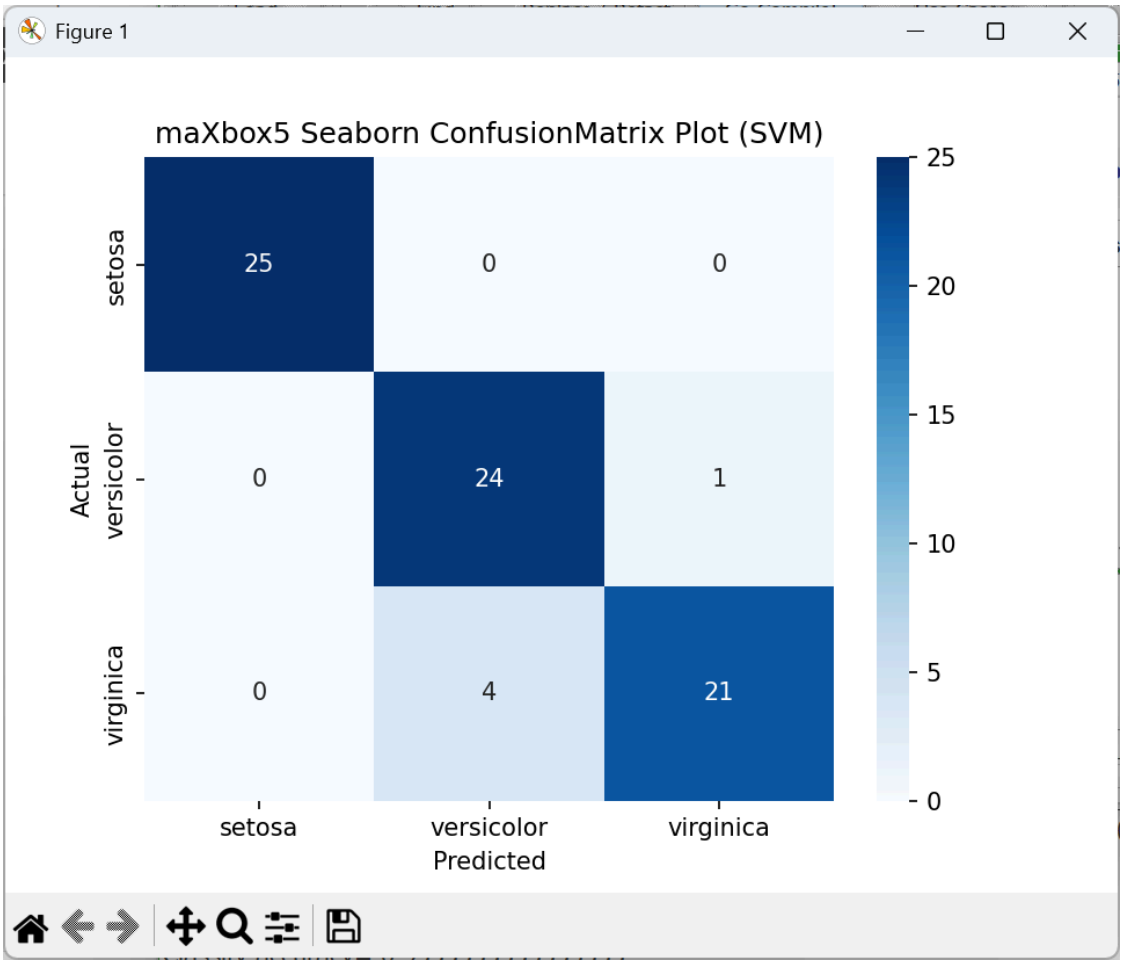
        // 5. Output prediction accuracy
        writeln('Classify accuracy='+evalstr('accuracy_score(y_test,
        execstr('print(classification_report(y_test, svm.predict(X_t
        println(StringReplace(evalstr('(output.getvalue().strip())')
                                                    LF,CRLF,[rfRepl

        // 6. Generate and render Seaborn confusion matrix plot
        execstr('import seaborn as sns');
        execstr('cm = pd.DataFrame(confusion_matrix(y_test, svm.pred
                'columns=iris.target_names, index=iris.target_names)
                'sns.heatmap(cm, annot=True, cmap=''Blues'')' + lf +
                'plt.title("maXbox5 Seaborn ConfusionMatrix Plot (SV
                'plt.xlabel("Predicted")' + lf +
                'plt.ylabel("Actual")' + lf +
                'plt.show()');
      except
        raiseError;
      finally
        Free;
      end;
    end;
  end;
```

Key Changes from Random Forest to SVM:

1. Library Import: Replaced `from sklearn.ensemble import RandomForestClassifier` with `from sklearn.svm import SVC`.
2. Model Instantiation & Fitting: Replaced `RandomForestClassifier(n_estimators=100, ...)` with `SVC(kernel='rbf', C=1.0, random_state=123456)` to fit a Radial Basis Function (RBF) kernel decision boundary.
3. Evaluation & Visualization: Maintained the exact maXbox evaluation flow, calling `accuracy_score` and plotting the `confusion_matrix` heatmap with Seaborn.

Advertisement
Privacy Settings



Confusion Matrix of Support Vector Machine

```
Classify accuracy= 0.9333333333333333
              precision recall    f1-score support

      0       1.000     1.000     1.000        25
      1       0.857     0.960     0.906        25
      2       0.955     0.840     0.894        25
```

```
maxcalcF('(24/28)') //precision versicolor
maxcalcF('(24/25)') //recall
maxcalcF('(21/22)') //precision virginica
maxcalcF('(21/25)') //recall
```

Advertisement
Privacy Settings



Classify 3 Locs as our next step



Classify with 4 Classes : Swiss, France, Germany and Belgian Locs

Advertisement
Privacy Settings

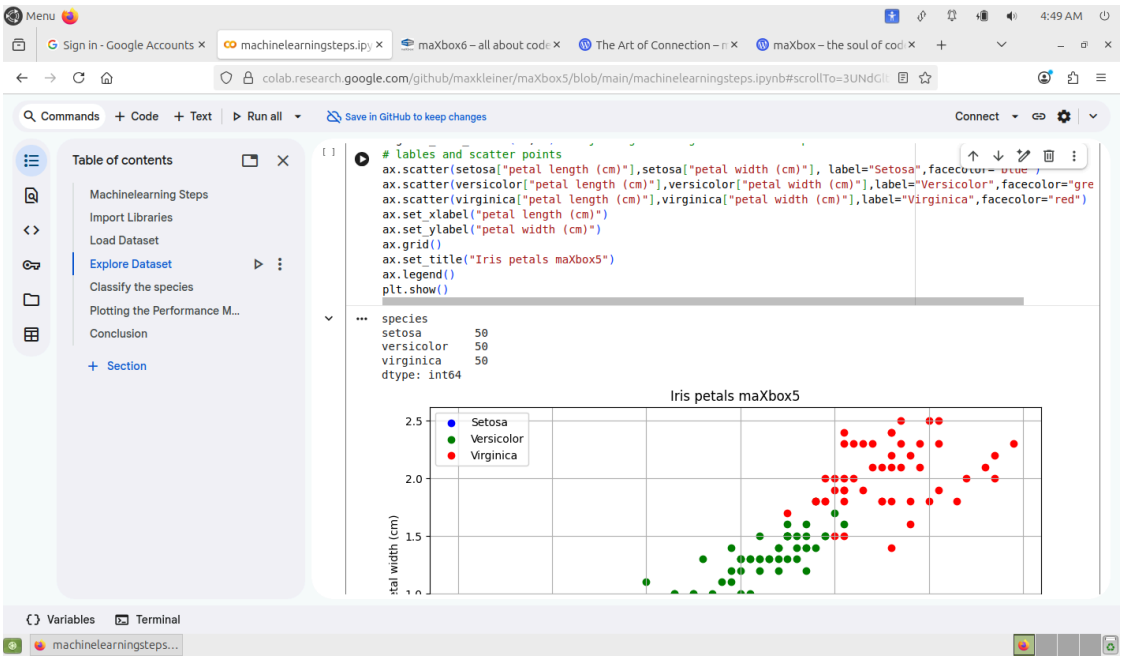
Update 1.2

The update add **classification_report** and data explorer image scatter plot.

Download 1452_machinelearningsteps2python12plotdataset.pas (maXbox5)

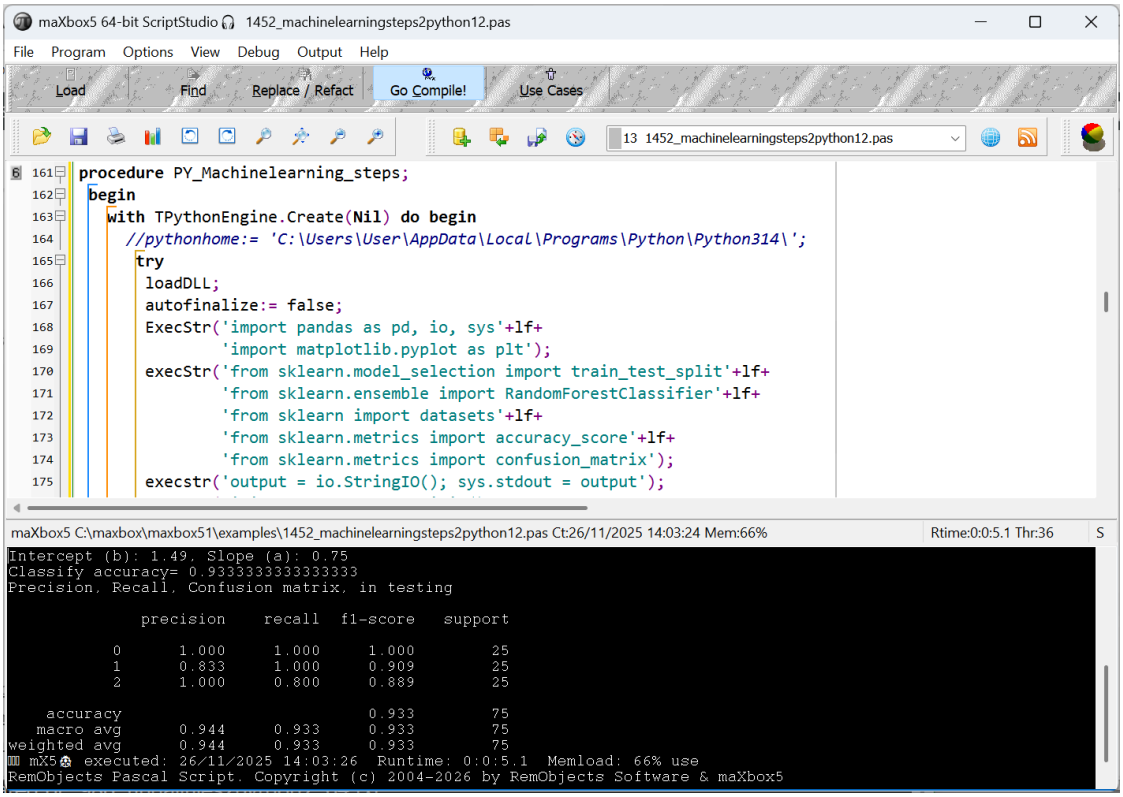
As a Jupyter Notebook

maXbox5/machinelearningsteps.ipynb at main · maxkleiner/maXbox5



Jupyter Notebook on Ubuntu Mate 24 Linux

Advertisement
Privacy Settings



Download 1452_machinelearningsteps2python12.pas (maXbox5)

Technical Adviser: Robert Rottermann, Lecturer: Dr. Silvia Rothen

Example with Python 3.14

Advertisement
Privacy Settings

Download 1452_iris_rf.py (maXbox5)

writ(GetDosOutput('py -3.14 C:\maxbox\maxbox51\examples\iris_rf.py –mode show',' C:\'));

```
# procedure PY_Machinelearning_steps; begin with TPythonEngine.Cr

# iris_rf.py

#!/usr/bin/env python3
"""
Train a RandomForest on the Iris dataset and output accuracy + co

Usage:
    python iris_rf_cli.py --mode show     # show plot (default)
    python iris_rf_cli.py --mode save    # save plot to confusion
    python iris_rf_cli.py --mode both    # save and show
    python iris_rf_cli.py --mode none    # no plot, just accuracy
"""

import argparse

import matplotlib
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

matplotlib.use("TkAgg")

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn import datasets
from sklearn.metrics import accuracy_score, confusion_matrix


def main(mode: str, outfile: str):
    # ------------------------------------------------------------
    # Load the Iris dataset
    # ------------------------------------------------------------
    iris = datasets.load_iris()
    df = pd.DataFrame(iris.data, columns=iris.feature_names)

    # ------------------------------------------------------------
    # Train/test split
    # ------------------------------------------------------------
    X_train, X_test, y_train, y_test = train_test_split(
        df[iris.feature_names],
        iris.target,
        test_size=0.5,
        stratify=iris.target,
        random_state=123456,
    )

    # ------------------------------------------------------------
    # Train Random Forest Classifier
    # ------------------------------------------------------------
    rf = RandomForestClassifier(
        n_estimators=100,
        oob_score=True,
        random_state=123456,
    )
    rf.fit(X_train, y_train)

    # ------------------------------------------------------------
    # Accuracy
```

```
    # ------------------------------------------------------
    y_pred = rf.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    print(f"Classification accuracy = {acc:.4f}")

    # ------------------------------------------------------
    # Confusion Matrix
    # ------------------------------------------------------
    if mode in ("show", "save", "both"):
        cm = pd.DataFrame(
            confusion_matrix(y_test, y_pred),
            index=iris.target_names,
            columns=iris.target_names,
        )

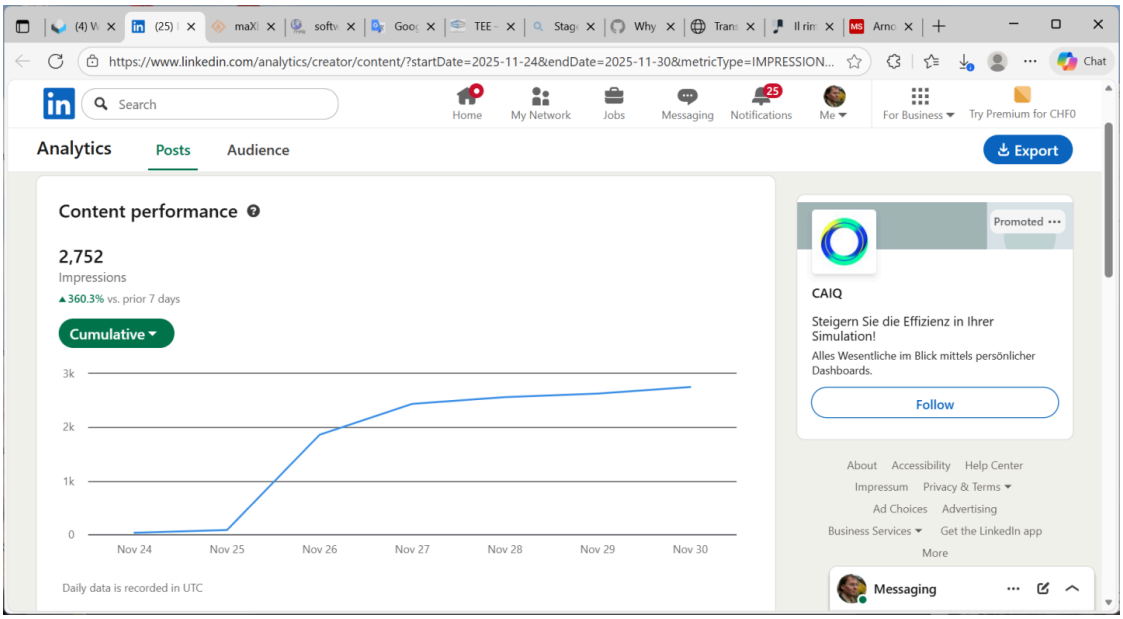
        plt.figure()
        sns.heatmap(cm, annot=True, cmap="Blues", fmt="d")
        plt.title("Confusion Matrix – Random Forest (Iris) by Rob
        plt.xlabel("Predicted")
        plt.ylabel("Actual")
        plt.tight_layout()

        if mode in ("save", "both"):
            plt.savefig(outfile, dpi=150, bbox_inches="tight")
            print(f"Confusion matrix saved to: {outfile}")

        if mode in ("show", "both"):
            plt.show()
        else:
            # Close the figure when not showing interactively
            plt.close()
    else:
        # mode == "none": no plot, just accuracy printed above
        pass

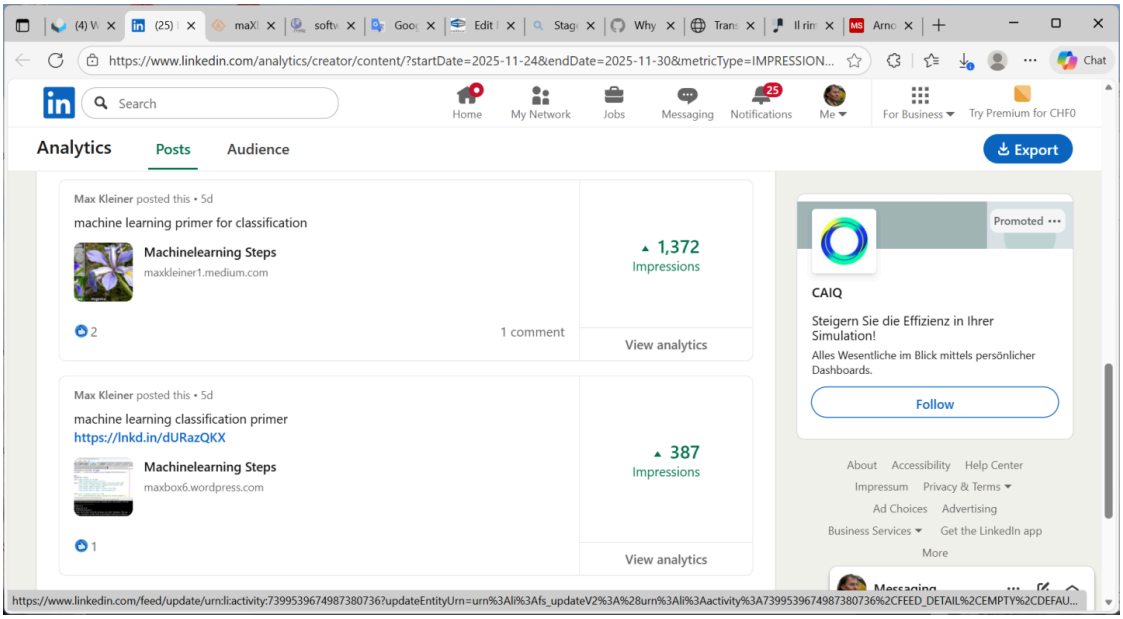

if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description="RandomForest on Iris with selectable output
    )
    parser.add_argument(
        "--mode",
        choices=["show", "save", "both", "none"],
        default="show",
        help="How to handle the confusion-matrix plot.",
    )
    parser.add_argument(
        "--outfile",
        default="confusion_matrix.png",
        help="Output file when mode is 'save' or 'both' (default:
    )

    args = parser.parse_args()
    main(mode=args.mode, outfile=args.outfile)
```



impact or reach

Advertisement
Privacy Settings



machinelearningsteps ahead

Upgrade App

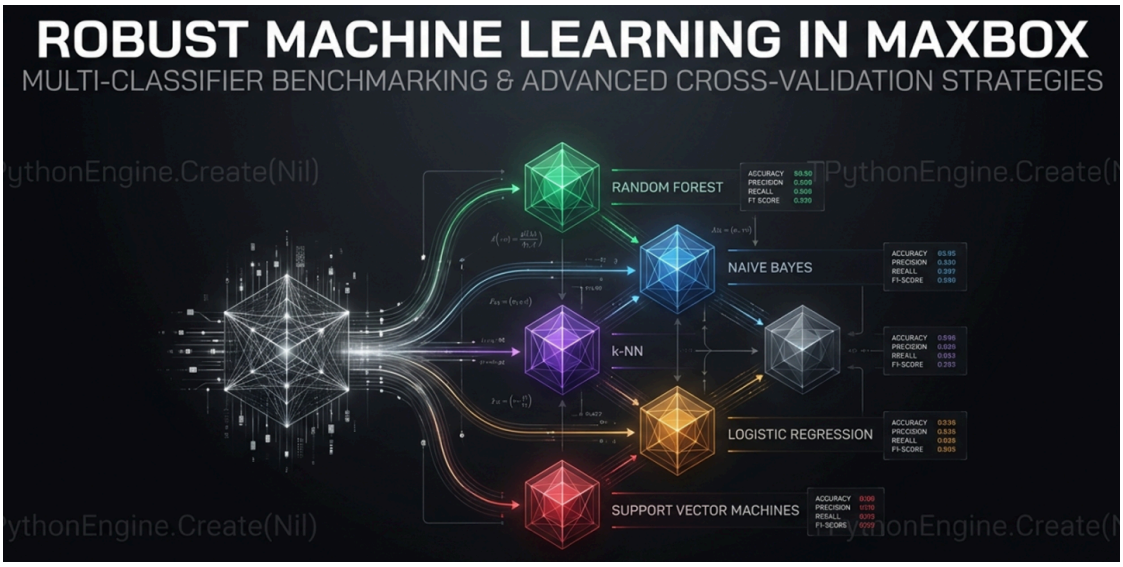
I have created and published the complete, ready-to-run maXbox script `ml_5_algorithms_visual_studio.pas` to your Studio panel.

This script implements an interactive visual environment in maXbox that executes and benchmarks 5 core Machine Learning algorithms using Python4Delphi integration (`TPythonEngine`) alongside native Pascal GUI controls and canvas plotting:

1. Random Forest Classifier (`RandomForestClassifier`): Supervised ensemble learning combining decision trees on multi-feature datasets.

2. Gaussian Naive Bayes (`GaussianNB`): Probabilistic classifier calculating class posterior probabilities $P(\text{Class}|X) \propto P(X|\text{Class})P(\text{Class})$ grounded in Bayes' Theorem.
3. k-Nearest Neighbors (`KNeighborsClassifier`): Distance-based instance classifier voting on majority class labels among k nearest neighbors.
4. Logistic Regression (`LogisticRegression`): Linear model establishing decision boundaries via sigmoidal probability estimation.
5. k-Means Clustering (`KMeans`): Unsupervised algorithm partitioning observations into k distinct clusters around iterative centroids.

Advertisement
Privacy Settings



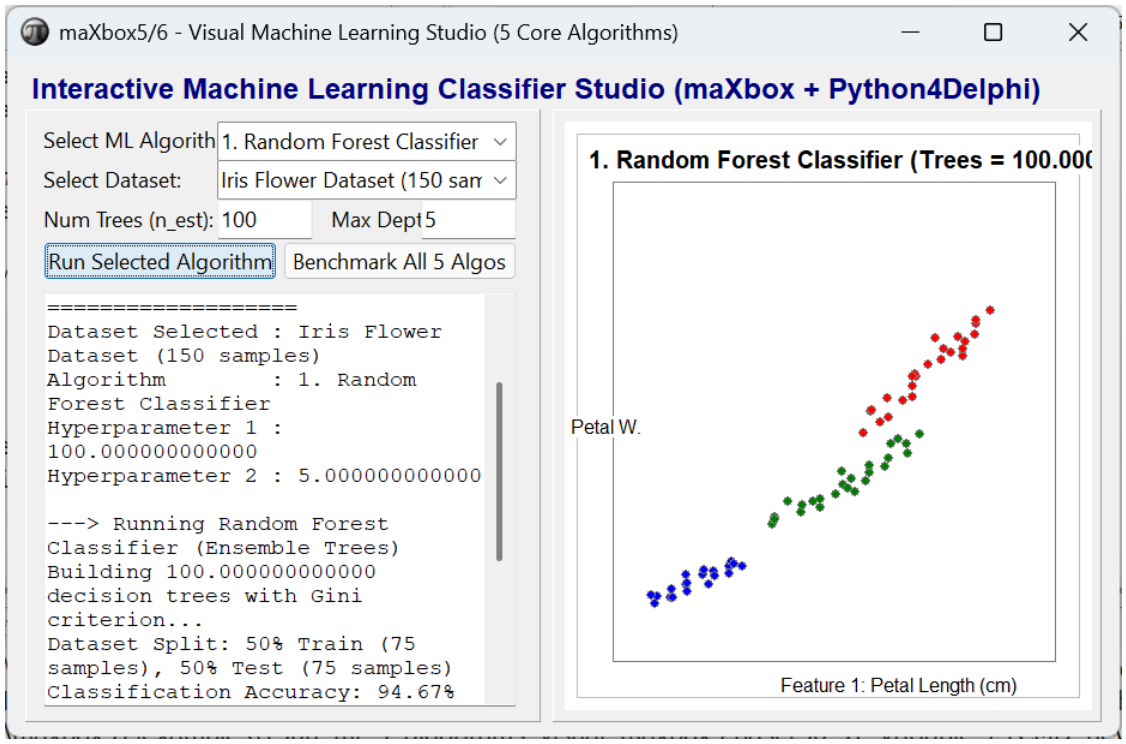
https://sourceforge.net/projects/maxbox5/files/EKON30/202_EKON30_The_Embedded_ML_Python_Pipeline_maXbox_Studio_Iristest.pdf/download

Script Capabilities:

- Interactive GUI Controls: Features dropdown selectors (`TComboBox`) for switching algorithms and datasets (Iris Flowers, Gaussian Clusters, Moons), plus dynamic hyperparameter input fields (`EdtParam1`, `EdtParam2`).
- Visual Canvas Plotter (`TImage`): Renders 2D feature scatter plots, linear decision boundaries, cluster centroids, or an Accuracy Benchmark Bar Chart across all 5 algorithms.
- Detailed Execution Logging: Outputs precision, recall, F1-scores, and confusion matrix breakdowns directly into a `TMemo` log view.

Advertisement
Privacy Settings

📊 Would you like to create a study guide report or interactive quiz summarizing how these 5 machine learning algorithms compare in classification performance?



https://sourceforge.net/projects/maxbox5/files/examples/1500_ml_5_algorithms_visual_maxbox5.pas/download

Evaluating a classifier involves three primary performance measures: the confusion matrix, recall, and precision.

Advertisement
Privacy Settings

1. Confusion Matrix

- What it measures: A grid that compares actual target classes against predicted classes to show exactly how many instances were correctly classified versus misclassified.
- How to interpret it: The diagonal cells represent correct predictions, while off-diagonal cells highlight where the model confuses one class for another (false positives and false negatives).

---

2. Recall (Sensitivity)

- What it measures: The ratio of actual positive instances that are truly detected by the classifier.
- Practical rule: Bad recall means failing to recognize something that is actually there.
- Example: In the Iris classification model, a Virginica recall of 80% means 80% of true Virginica flowers were correctly detected, while 20% (5 samples) were missed and misclassified as Versicolor.

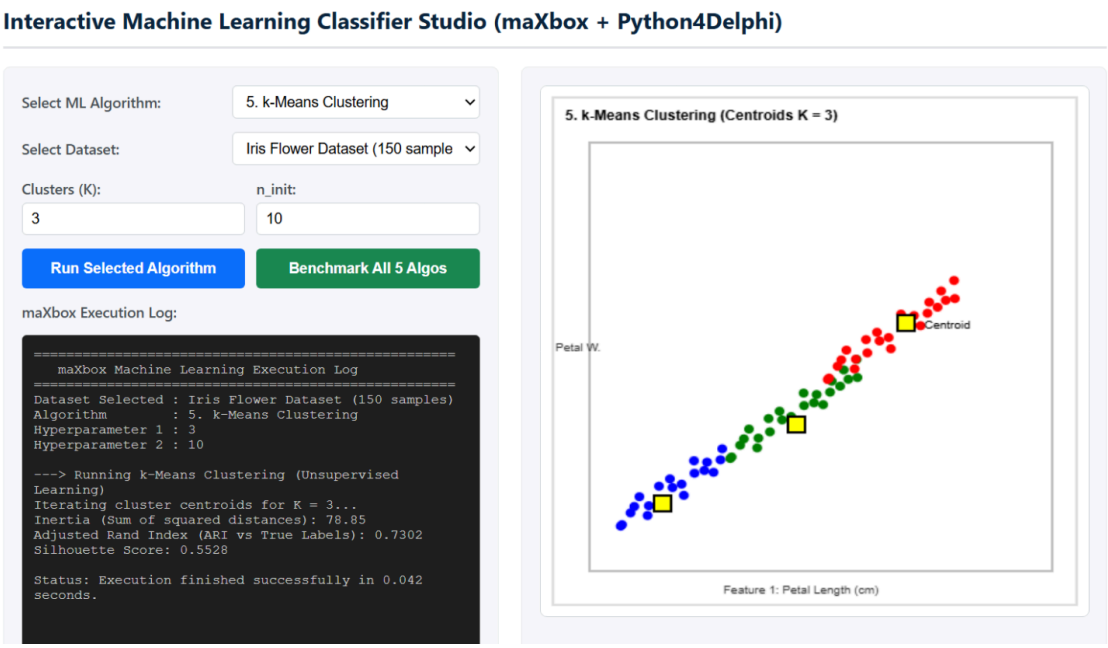
---

3. Precision (Specifity- Positive Predictive Value)

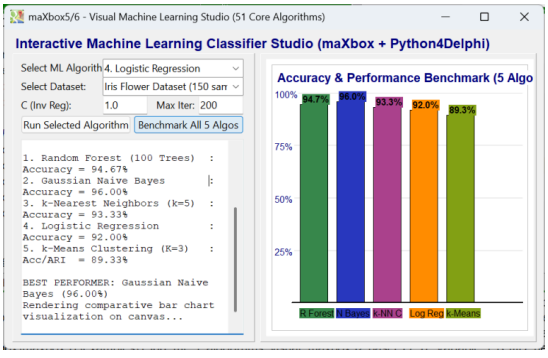
- What it measures: The accuracy of positive predictions—i.e., out of all instances predicted as a specific class, how many actually belong to it.
- Practical rule: Bad precision means confusing one thing for another.
- Example: The Versicolor precision drops to 83.3% because 5 Virginica samples were mistakenly labeled as Versicolor, resulting in 25 true Versicolors out of 30 total predicted Versicolors (\(25 / 30 = 0.833\)).

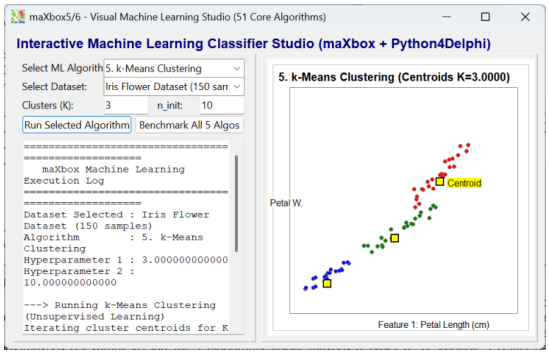
Advertisement
Privacy Settings

💡 Would you like me to generate a classification report or a visual confusion matrix plot using Python4Delphi in maXbox to test these metrics on a custom dataset?

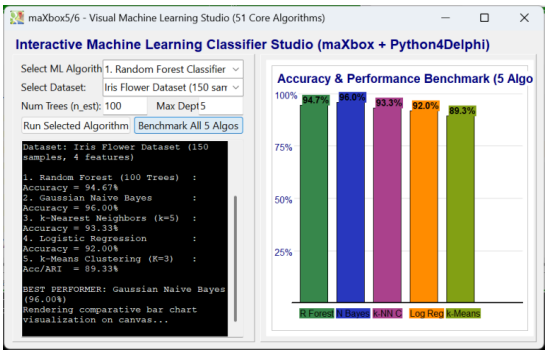


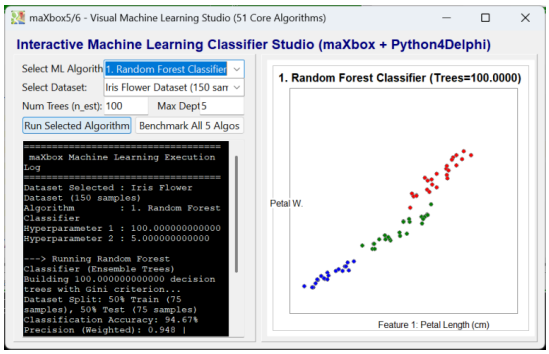
maXbox Visual Machine Learning Studio (5 Core Algorithms)





https://sourceforge.net/projects/maxbox5/files/examples/1500_ml_5_algorithms_visual_maxbox53.pas/downloa





https://sourceforge.net/projects/maxbox5/files/examples/1500_ml_5_algorithms_visual_maxbox54.pas/downloa

Advertisement

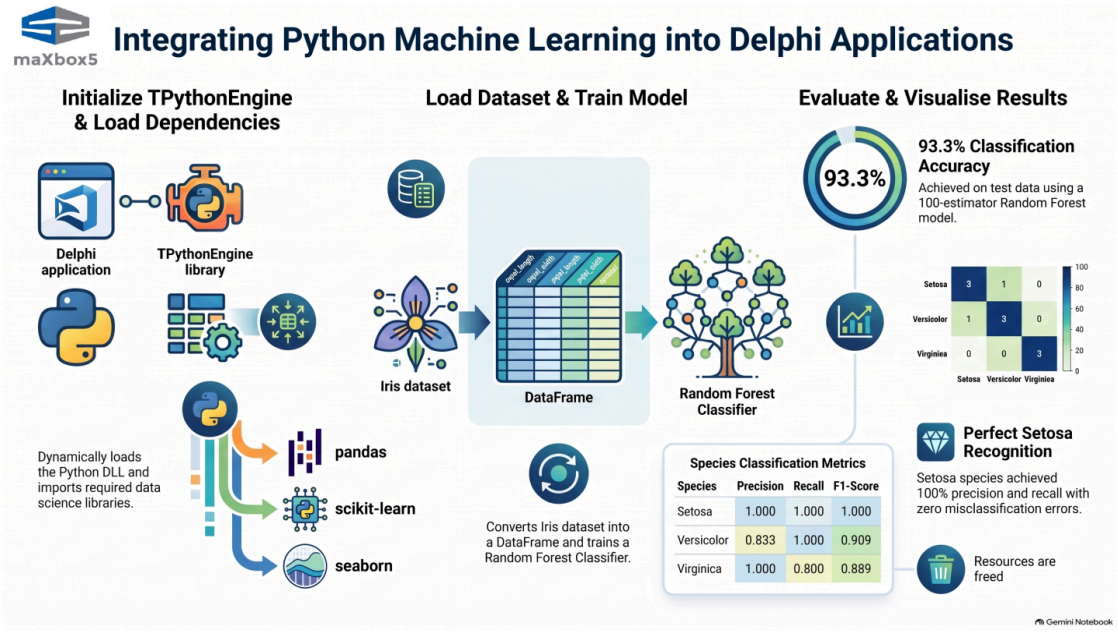


Who Do You Really Trust?

Belief isn't about facts but who says them. Learn how environment and trust shape what you think is real.

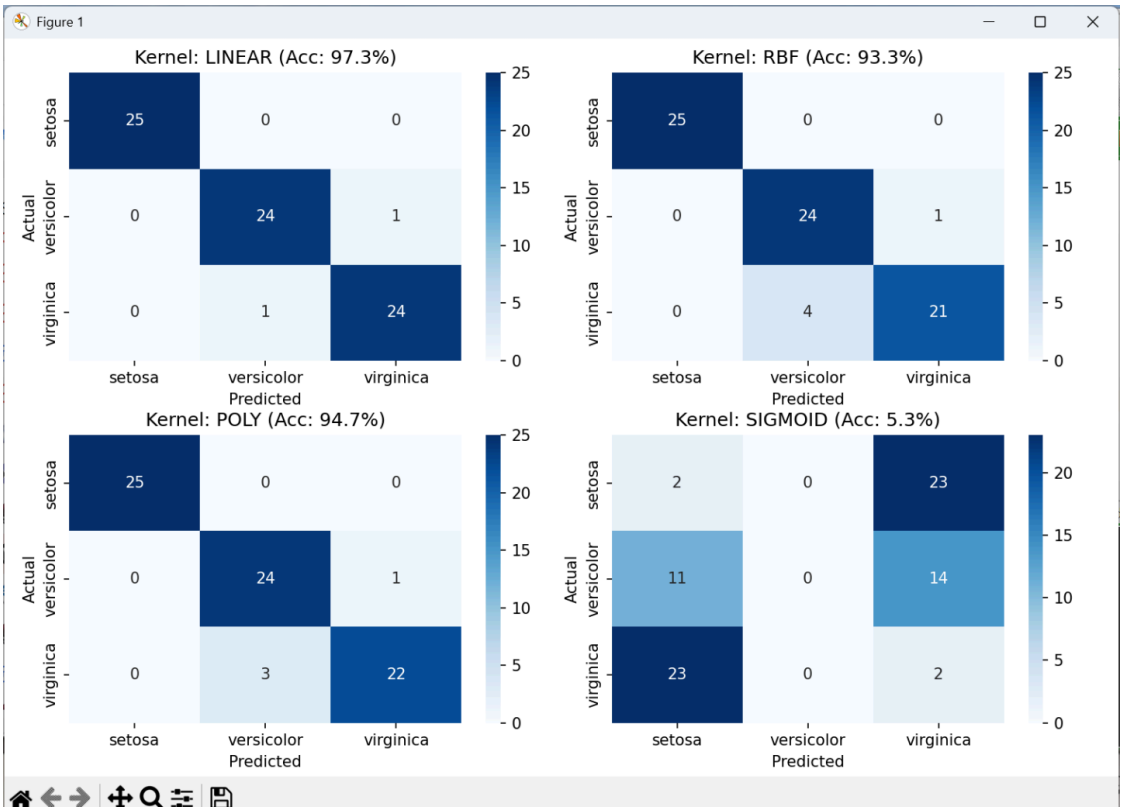
See how belief works

Privacy Settings



P4D Pipe

SVM Kernel Comparison



https://sourceforge.net/projects/maxbox5/files/examples/1500_ml_5_algorithms_visual_maxbox55.pas/download

```
=== SVM KERNEL SIDE-BY-SIDE BENCHMARK ===
Kernel  Accuracy Precision Recall F1-Score
linear  0.973333 0.973333 0.973333 0.973333
rbf     0.933333 0.937229 0.933333 0.933092
poly    0.946667 0.948470 0.946667 0.946581
sigmoid 0.053333 0.035613 0.053333 0.042691
```

Advertisement



When Luck Held On

You fell in water once—did fate or luck save you? Discover the quiet moments shaping survival.

Read the story

Privacy Settings

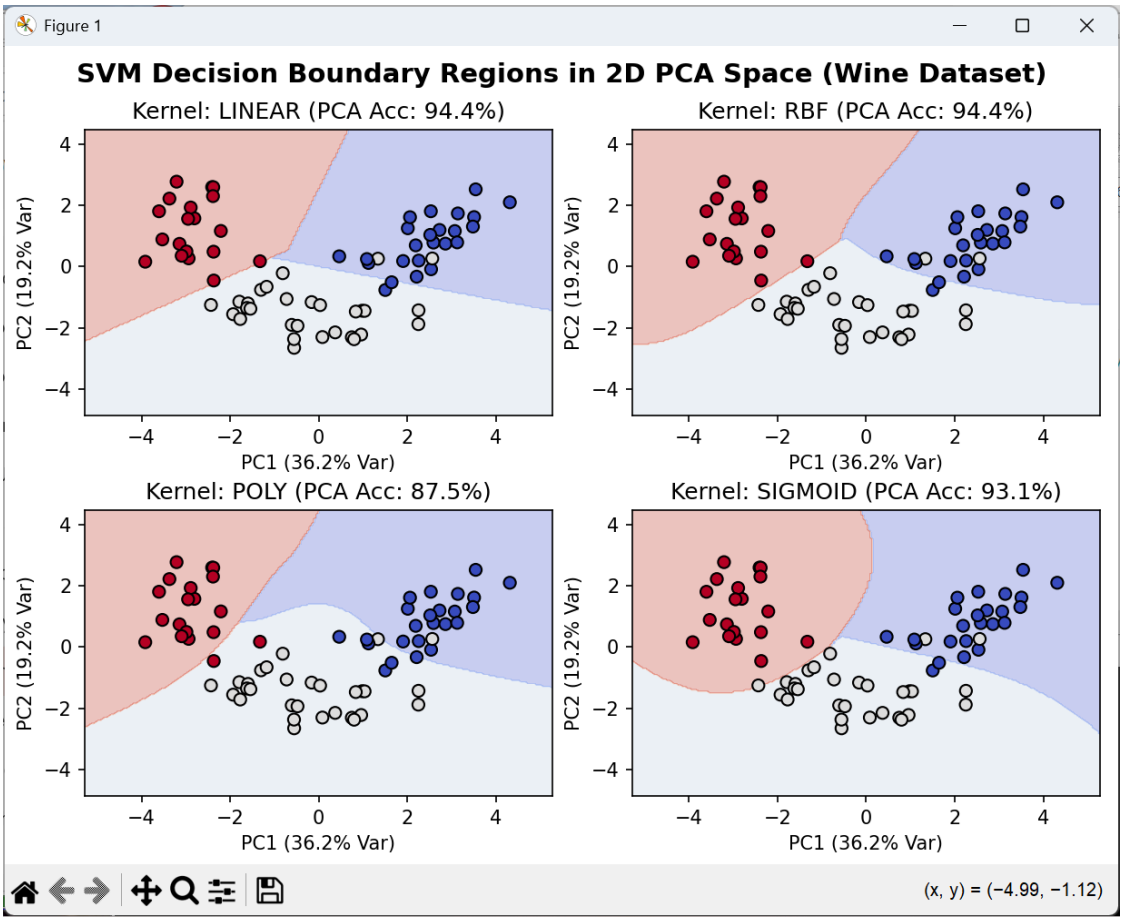
Key Differences Across Kernels:

- Linear Kernel (‘linear’): Works exceptionally well when feature spaces are linearly separable, providing high accuracy with low computational overhead [16].

- RBF Kernel ('rbf'): Uses Gaussian radial basis functions to construct smooth non-linear decision boundaries around target clusters.
- Polynomial Kernel ('poly'): Maps features into higher-dimensional polynomial feature spaces (default degree 3), effective for curved relationships.
- Sigmoid Kernel ('sigmoid'): Emulates artificial neural network activation behavior, though sensitive to scaling on smaller datasets.

⚙️ Would you like to add hyperparameter tuning controls (such as adjusting the regularization parameter C or gamma) to explore how they affect boundary smoothness and accuracy in maXbox?

PCM Boundaries



https://sourceforge.net/projects/maxbox5/files/examples/1500_ml_5_algorithms_visual_maxbox56.pas/download

Advertisement
Privacy Settings

Key Highlights of the PCA Boundary Pipeline:

- Feature Projection (PCA(n_components=2)): Reduces the 13 wine dimensions down to two orthogonal axes (PC1 and PC2), preserving ~55% of total dataset variance in a 2D plane.

- Continuous Decision Regions (plt.contourf): Generates predictions across a dense grid of PC1 and PC2 values to visually highlight how Linear, RBF, Polynomial, and Sigmoid boundaries separate the three wine classes.
- Test Sample Overlays: Plots actual test points over the decision regions so you can immediately see which samples fall on the correct side of the boundary versus those misclassified near the margins.
- This interactive web studio embeds a Sequential Minimal Optimization (SMO) Support Vector Machine solver directly in JavaScript. It allows you to drag sliders for C, γ (Gamma), Degree, and switching kernels in real time to observe decision boundaries, margins, and support vectors updating live on an HTML5 canvas:

Java Script Interactive Demo

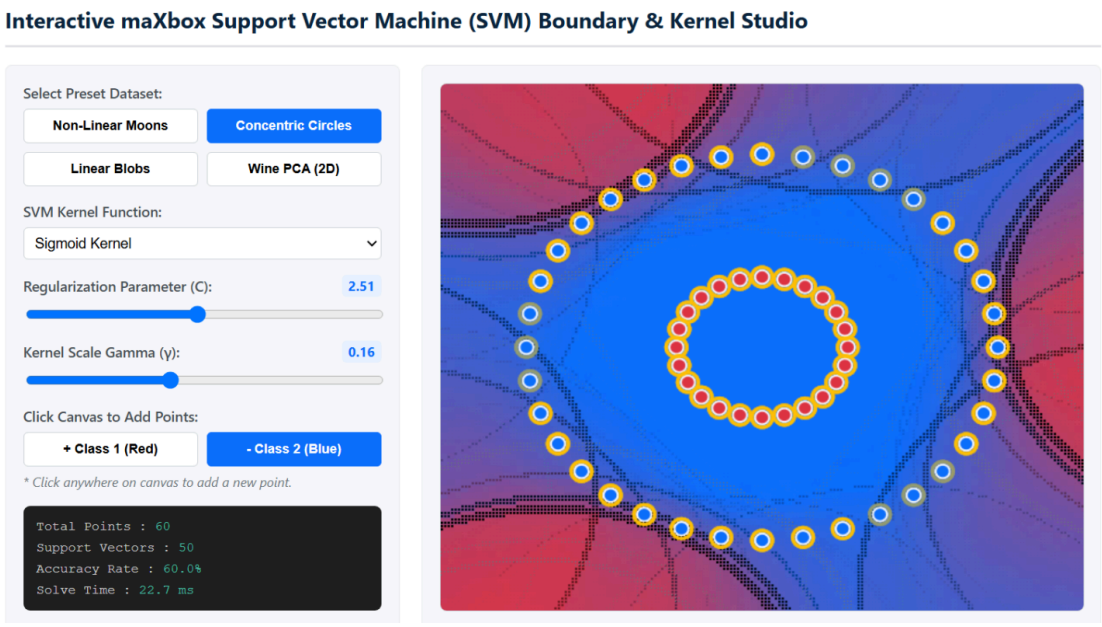
Studio Capabilities & Features:

1. Real-Time Canvas Contour Shader: Uses an embedded Sequential Minimal Optimization (SMO) solver running in JavaScript to calculate decision values $f(x) = \sum \alpha_i y_i K(x_i, x) + b$ continuously across the 2D plane.

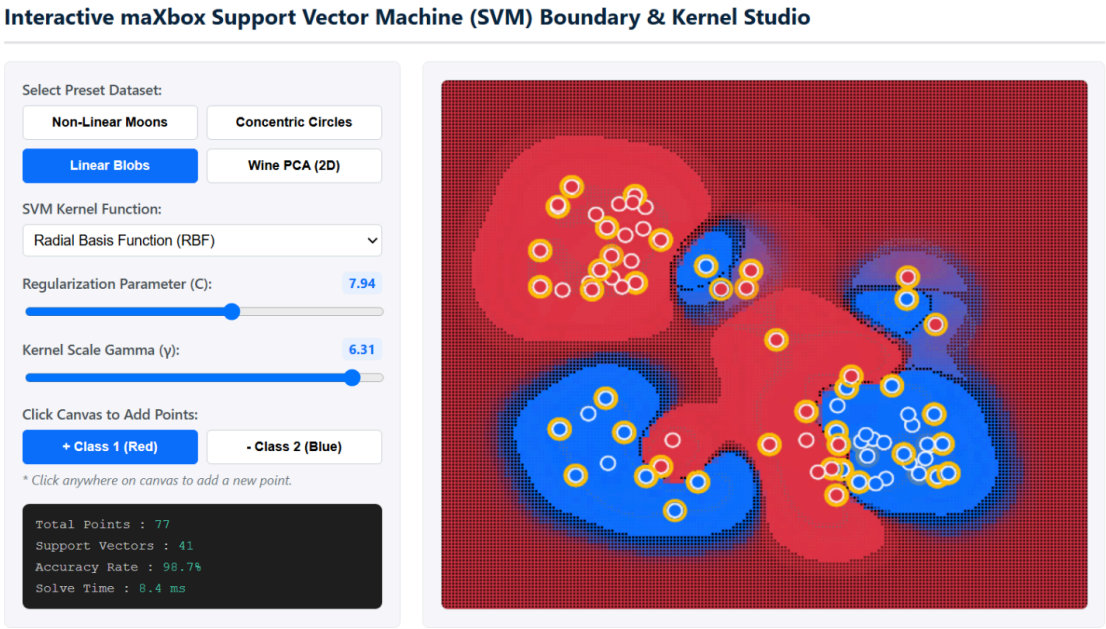
values $f(x) = \sum \alpha_i y_i K(x_i, x) + b$ continuously across the 2D plane.

1. values $f(x) = \sum \alpha_i y_i K(x_i, x) + b$ continuously
2. Interactive Parameter Tuning:
    - Regularization (C): Drag C to see the trade-off between margin width and classification errors.
    - Kernel Selection: Switch between RBF, Linear, Polynomial, and Sigmoid kernels.
    - Scale (γ) & Degree (d): Adjust Gamma or Polynomial Degree to see decision boundary curvature scale live.
3. Support Vector Highlighting: Support vectors ($\alpha_i > 0$) are highlighted with glowing yellow rings.
4. Preset Datasets & Click-to-Add Mode: Test on non-linear moons, concentric circles, linear blobs, or click anywhere on the canvas to add custom positive/negative points.

Advertisement
Privacy Settings

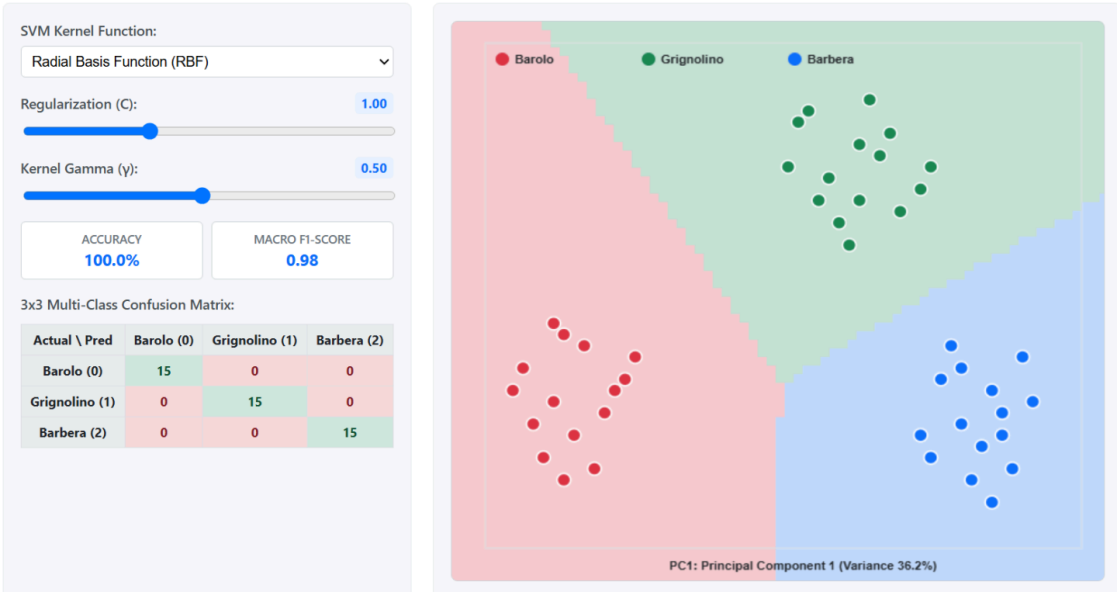


https://raw.githack.com/maxkleiner/maXbox5/main/assets/machinelearning52.htm



Advertisement
Privacy Settings

This web studio embeds the Wine dataset (2D PCA projection) into an interactive HTML5 canvas, providing real-time controls for SVM decision boundary regions, a feature scatter plot, and a 3×3 Confusion Matrix:



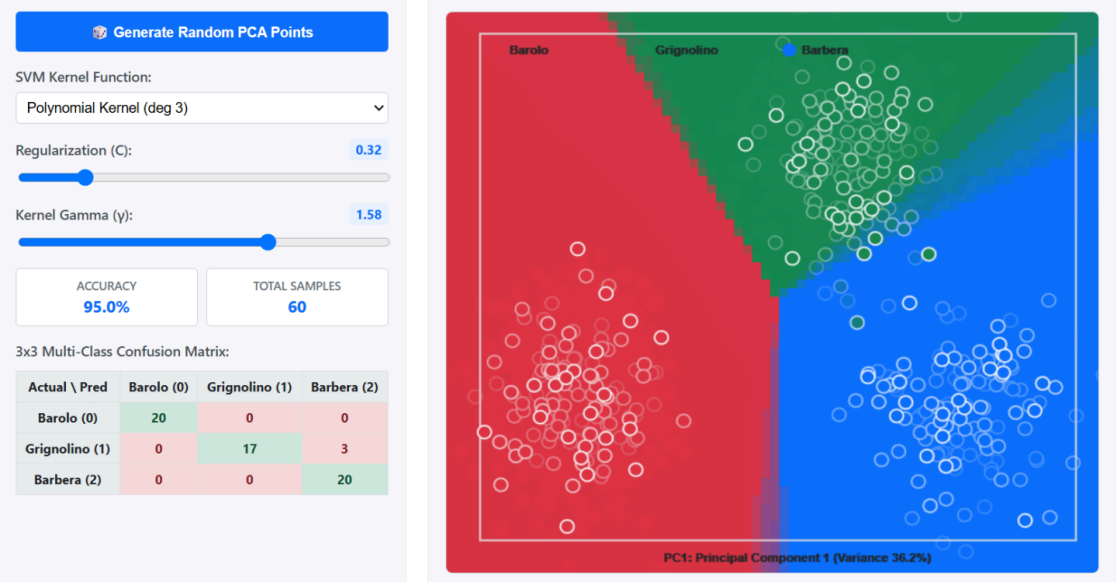
https://raw.githack.com/maxkleiner/maXbox5/main/assets/machinelearning51.htm

Here is the updated HTML, CSS, and JavaScript code containing a **randomized 2D PCA data generator** for the three Wine classes (**Barolo**, **Grignolino**, and **Barbera**).

This version includes a **"Generate Random PCA Points"** button that utilizes the **Box-Muller Gaussian transformation** to continuously resample synthetic 2D PCA cluster points around realistic feature centroids (`(-2.1, -0.9)`, `(0.8, 1.2)`, and `(2.0, -1.1)`), automatically recalculating the **3×3 confusion matrix**, **decision boundaries**, and **accuracy metrics** live.

Advertisement
Privacy Settings

Wine Dataset ML Studio – Scatter Plot & Confusion Matrix



Highlights of the Updated Core Code:

- **Box-Muller Gaussian Sampling (randomGaussian)**: Generates normal distributions for **PC1** and **PC2** centered around the true PCA cluster coordinates

for Barolo, Grignolino, and Barbera.

- Dynamic Resampling: Clicking "🎲 Generate Random PCA Points" creates 60 new randomized points across the 3 wine classes to test how well the SVM generalizes across different synthetic distributions.
- Live Confusion Matrix Updating: The 3×3 confusion matrix and overall classification accuracy update instantly as new points are sampled or when hyperparameters (C and γ) are tuned.

Advertisement
Privacy Settings

Bayes Core

Wie gross ist die Wahrscheinlichkeit nach Bayes Theorem dass ein poetischer Brief von einem Buchhändler stammt, wenn es 4 Buchhändler und 24 Bauern gibt?

According to Bayes' theorem, what is the probability that a poetic letter comes from a bookseller, given that there are 4 booksellers and 24 farmers?

Wenn der Brief zufällig von einer der 28 Personen stammt und „poetisch" keine zusätzliche Information über den Verfasser liefert, beträgt die Wahrscheinlichkeit, dass er von einem Buchhändler stammt:

:

$$P(\text{Buchhändler}) = \frac{4}{4+24} = \frac{4}{28} = \frac{1}{7} \approx 14{,}3\%.$$

P(Buchha¨ndler)=4+244=284=71≈14,3%.

```
P(\text{Buchhändler})=\frac{4}{4+24}=\frac{4}{28}=\frac17\approx 14{
```

Mit Bayes

Sei B: „Verfasser ist Buchhändler" und P: „Brief ist poetisch". Nach dem Satz von Bayes gilt:

```
P(B\mid P)=\frac{P(P\mid B)\,P(B)}{P(P\mid B)\,P(B)+P(P\mid \text{Bau
```

Ohne Angaben, wie wahrscheinlich Buchhändler bzw. Bauern poetische Briefe schreiben, ist die naheliegende Annahme:

```
P(P\mid B)=P(P\mid \text{Bauer}).
```

Dann kürzt sich die poetische Eigenschaft heraus, und es bleibt:

```
P(B\mid P)=P(B)=\frac{4}{28}=\frac17\approx14{,}3\%.
```

Das Bayes-Theorem verknüpft die Wahrscheinlichkeit der Hypothese mit der Wahrscheinlichkeit der Beobachtung unter dieser Hypothese; ohne unterschiedliche likelihoods verändert das Merkmal „poetisch“ die a-priori-Wahrscheinlichkeit nicht. [ethz][studysmarter]

Advertisement
Privacy Settings

Antwort: `\frac17`, also etwa 14,3%.

Likelihood bedeutet hier: *Wie wahrscheinlich ist die Beobachtung „der Brief ist poetisch“, wenn der Verfasser jeweils ein Buchhändler oder ein Bauer ist?* Sie bewertet also, wie gut jede mögliche Erklärung die beobachtete Eigenschaft erklärt. [quant-expert][ethz]

Im Brief-Beispiel

Sei `P` die Beobachtung „der Brief ist poetisch“ und `B` die Hypothese „der Verfasser ist Buchhändler“. Dann ist

```
P(P\mid B)
```

die Likelihood für Buchhändler: die Wahrscheinlichkeit, dass ein Brief poetisch ist, *unter der Annahme*, dass er von einem Buchhändler stammt.

Entsprechend ist

```
P(P\mid \text{Bauer})
```

die Likelihood für Bauern: die Wahrscheinlichkeit eines poetischen Briefes, *wenn der Verfasser ein Bauer ist*.

Warum sie wichtig ist

Im Bayes-Theorem wird die Likelihood mit der a-priori-Wahrscheinlichkeit multipliziert:

```
P(B\mid P)=\frac{P(P\mid B)\,P(B)}{P(P\mid B)\,P(B)+P(P\mid \text{Bau
```

Die Likelihood entscheidet, ob der poetische Stil die Vermutung „Buchhändler“ stärker oder schwächer macht.[quant-expert]

Advertisement
Privacy Settings

Beispiel

Hypothese	Likelihood eines poetischen Briefs	Bedeutung
Buchhändler	0,8	Buchhändler schreiben zu 80% poetische Briefe
Bauer	0,2	Bauern schreiben zu 20% poetische Briefe

Dann wäre ein poetischer Brief ein starkes Indiz für einen Buchhändler, denn `0{,}8/0{,}2=4`: Die Daten passen viermal besser zur Hypothese „Buchhändler“.[ethz]

Im früheren Beispiel hatten wir dagegen angenommen:

```
P(P\mid B)=P(P\mid \text{Bauer}).
```

Das heißt: Die Likelihoods sind gleich. Der poetische Stil liefert dann keine zusätzliche Information, und es bleibt nur die a-priori-Wahrscheinlichkeit `4/28=1/7\approx14{,}3\%`. Eine Likelihood ist dabei nicht automatisch die Wahrscheinlichkeit der Hypothese selbst; sie misst die Plausibilität der Beobachtung *gegeben* die Hypothese.[quant-expert][datacamp]

The probability depends on how likely each group is to write a “poetic letter.” Bayes’ theorem combines that likelihood with the base rates (4 booksellers vs. 24 farmers) to give the posterior probability.[en.wikipedia][investopedia]

Advertisement
Privacy Settings

Bayes’ theorem setup

Let:

- `B` = “the letter comes from a bookseller”
- `F` = “the letter comes from a farmer”
- `P` = “the letter is poetic”

You’re given the prior probabilities from the population counts:

You’re given the prior probabilities from the population counts:

$$P(B) = \frac{4}{4+24} = \frac{4}{28} = \frac{1}{7}, \quad P(F) = \frac{24}{28} = \frac{6}{7}.$$

P(B)=4+244=284=71,P(F)=2824=76.

```
P(B) = \frac{4}{4+24} = \frac{4}{28} = \frac{1}{7}, \quad
P(F) = \frac{24}{28} = \frac{6}{7}.
```

Bayes' theorem says:

Bayes' theorem says:

$$P(B \mid P) = \frac{P(P \mid B)\,P(B)}{P(P \mid B)\,P(B) + P(P \mid F)\,P(F)}.$$

P(B|P)=P(P|B)P(B)+P(P|F)P(F)P(P|B)P(B).

```
P(B \mid P) = \frac{P(P \mid B)\,P(B)}{P(P \mid B)\,P(B) + P(P \mid I
```

[en.wikipedia]

Advertisement



Who Do You Really Trust?

Belief isn't about facts but who says them. Learn how environment and trust shape what you think is real.

See how belief works

Privacy Settings

What's missing: the likelihoods

To get a numeric answer you need:

- `P(P \mid B)`: probability a bookseller writes a poetic letter
- `P(P \mid F)`: probability a farmer writes a poetic letter

Without these, the result can't be uniquely determined. Different assumptions give very different answers.

Example calculations

1. If both groups are equally poetic
   Suppose `P(P \mid B) = P(P \mid F) = p`. Then:`P(B \mid P) = \frac{p \cdot \frac{1}{7}}{p \cdot \frac{1}{7} + p \cdot \frac{6}{7}} = \frac{1/7}{1/7 + 6/7} = \frac{1}{7} \approx 14.3\%.` In this special case, the "poetic" clue doesn't change the prior; the answer is just the base rate. [x-team][samuel-book.github]
2. If booksellers are more poetic
   Suppose booksellers are 3× as likely to write poetic letters as farmers:

$P(P \mid B) = 3q, P(P \mid F) = q$. Then:$P(B \mid P) = \frac{3q \cdot \frac{1}{7}}{3q \cdot \frac{1}{7} + q \cdot \frac{6}{7}} = \frac{3/7}{3/7 + 6/7} = \frac{3}{9} = \frac{1}{3} \approx 33.3\%.$

3. If only booksellers write poetic letters
   Suppose $P(P \mid F) = 0$ and $P(P \mid B) > 0$. Then:$P(B \mid P) = \frac{P(P \mid B)\cdot \frac{1}{7}}{P(P \mid B)\cdot \frac{1}{7} + 0} = 1.$ Any poetic letter must come from a bookseller.

Advertisement
Privacy Settings

Bottom line

With only "4 booksellers and 24 farmers," Bayes' theorem gives:

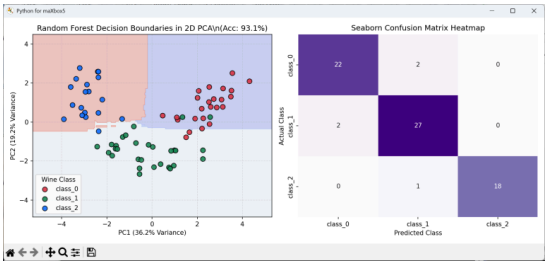
$$P(B \mid P) = \frac{P(P \mid B) \cdot \frac{1}{7}}{P(P \mid B) \cdot$$

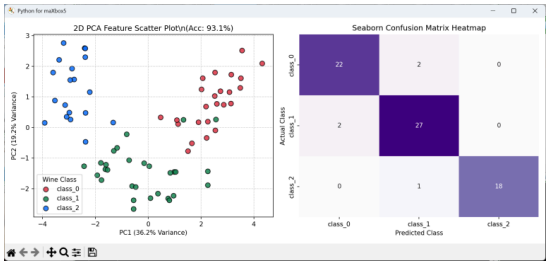
and you need the two likelihoods $P(P \mid B)$ and $P(P \mid F)$ to compute a specific number.[en.wikipedia][investopedia]

The Wine Dataset 🍷

What Was Updated:

- 2D Meshgrid Contours (plt.contourf): Generates a dense grid across PC1 and PC2 values to plot filled contour decision regions, showing exactly how the ensemble of decision trees partitions the feature space into territories for Barolo, Grignolino, and Barbera wines.
- Overlaid Sample Scatter Plot: Displays test sample points over the colored decision regions so you can visually verify which data points lie safely within class boundaries versus those near classification margins.
- Integrated Heatmap: Preserves the side-by-side Seaborn confusion matrix heatmap to compare visual boundary regions directly against numerical classification counts.





Advertisement

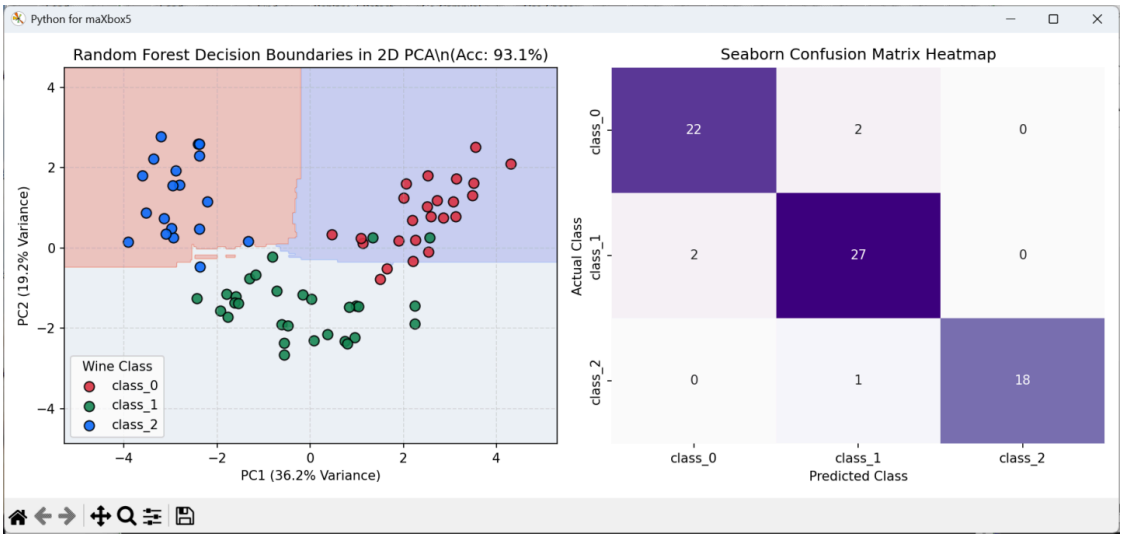


When Luck Held On

You fell in water once—did fate or luck save you? Discover the quiet moments shaping survival.

Read the story

Privacy Settings



https://sourceforge.net/projects/maxbox5/files/examples/1501_wine_dataset_ml_studio1.pas/download

	precision	recall	f1-score	support
0	0.917	0.917	0.917	24
1	0.900	0.931	0.915	29
2	1.000	0.947	0.973	19
accuracy			0.931	72

macro avg 0.939 0.932 0.935 72
weighted avg 0.932 0.931 0.931 72

Advertisement

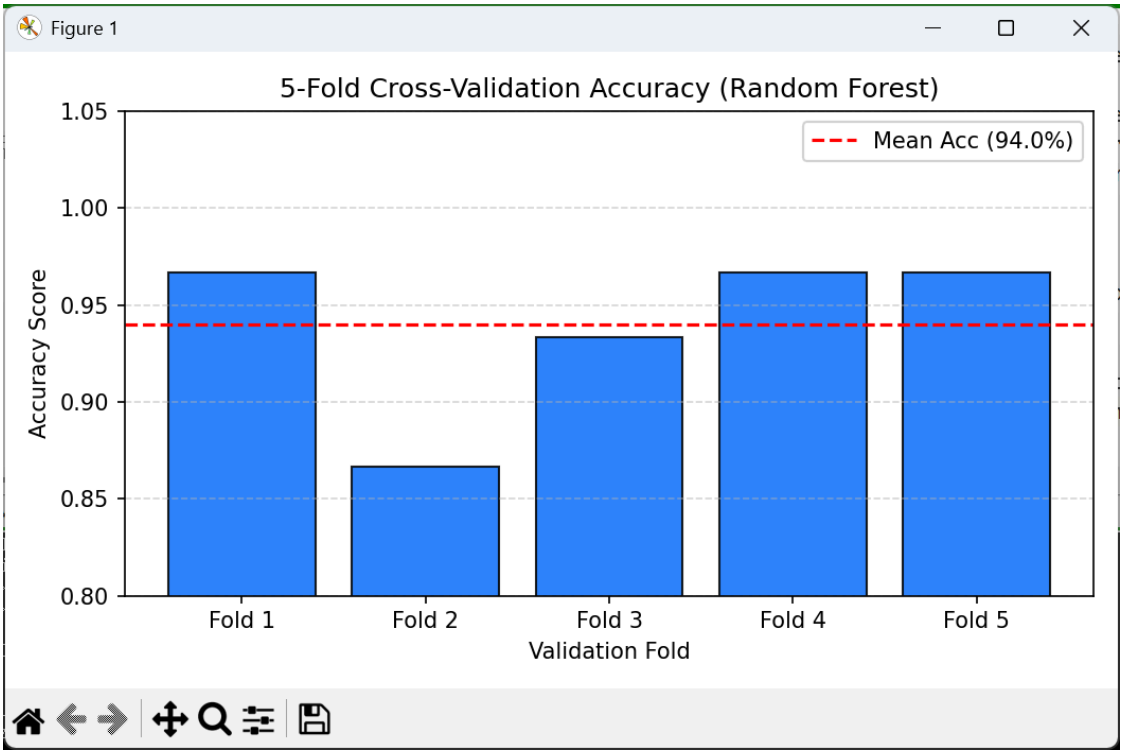


Bathroom Upgrade Made Easy

Transform your shower with a double-rod curtain setup for a custom, designer look without renovation.

See the full guide

Privacy Settings



Key Workflow Highlights:

1. StratifiedKFold: Ensures each of the 5 folds retains the same proportion of target flower species (Setosa, Versicolor, Virginica) as the complete dataset.
2. cross_val_score: Automatically handles model cloning, fold fitting, prediction, and accuracy scoring across all 5 iterations.
3. Variance Evaluation: `scores.mean()` calculates the expected generalization accuracy, while `scores.std()` measures how sensitive the model is to different training subsets.

Advertisement



Bathroom Upgrade Made Easy

Transform your shower with a double-rod curtain setup for a custom, designer look without renovation.

See the full guide

Privacy Settings

To calculate and visualize an aggregated confusion matrix across all folds during K-fold cross-validation, use scikit-learn's cross_val_predict1.

Instead of evaluating a single test split23, cross_val_predict generates out-of-fold predictions for every sample in the dataset while ensuring no model sees its test fold during training1. Passing these predictions into confusion_matrix produces an accumulated confusion matrix14.

How It Works:

1. Out-of-Fold Predictions (cross_val_predict): Iterates through each of the 5 folds. For each fold, the classifier is trained on 80% of the data and predicts the remaining 20%.
2. Aggregated Matrix (confusion_matrix(y, y_pred_cv)): Computes the true positives, false positives, and false negatives across all 150 samples in the Iris dataset.
3. Visual Heatmap (sns.heatmap): Renders the annotated matrix showing exact sample counts per class (Setosa, Versicolor, Virginica).

5 Fold Cross Validation Confusion Matrix (5CVCM)

=== 5-FOLD CROSS-VALIDATION CLASSIFICATION REPORT ===
features: ['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash', 'magnesium', 'total_phenols', 'flavanoids', 'nonflavanoid_phenols', 'proanthocyanins', 'color_intensity', 'hue', 'od280/od315_of_diluted_wines', 'proline']
precision recall f1-score support

```
           0       0.917     0.917     0.917        24
           1       0.900     0.931     0.915        29
           2       1.000     0.947     0.973        19

accuracy                             0.931        72
```

Advertisement



Struggling to Focus?

If starting tasks feels impossible, discover how small steps can make your ADHD days manageable.

See Small Steps

Privacy Settings

macro avg 0.939 0.932 0.935 72 weighted avg 0.932 0.931 0.931 72

```
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        50
  versicolor       0.90      0.92      0.91        50
   virginica       0.92      0.90      0.91        50
    accuracy                           0.94       150
```

macro avg 0.94 0.94 0.94 150 weighted avg 0.94 0.94 0.94 150

```
              precision    recall  f1-score   support
      setosa       1.00      1.00      1.00        50
  versicolor 0.90 0.92 0.91 50 virginica 0.92 0.90 0.91 50
    accuracy                           0.94       150
```

macro avg 0.94 0.94 0.94 150 weighted avg 0.94 0.94 0.94 150

```
              precision    recall  f1-score   support
      setosa       1.00      1.00      1.00        50
  versicolor   0.90 0.92 0.91 50 virginica 0.92 0.90 0.91 50
    accuracy                           0.94       150
```

macro avg 0.94 0.94 0.94 150 weighted avg 0.94 0.94 0.94 150

```
              precision    recall  f1-score   support
      setosa       1.00      1.00      1.00        50
  versicolor 0.90 0.92 0.91 50 virginica 0.92 0.90 0.91 50
    accuracy                           0.94       150
```

Advertisement
Privacy Settings

macro avg 0.94 0.94 0.94 150 weighted avg 0.94 0.94 0.94 150

```
              precision    recall  f1-score   support
  setosa       1.00      1.00      1.00        50
  versicolor 0.90 0.92 0.91 50 virginica 0.92 0.90 0.91 50
  accuracy                           0.94       150
```

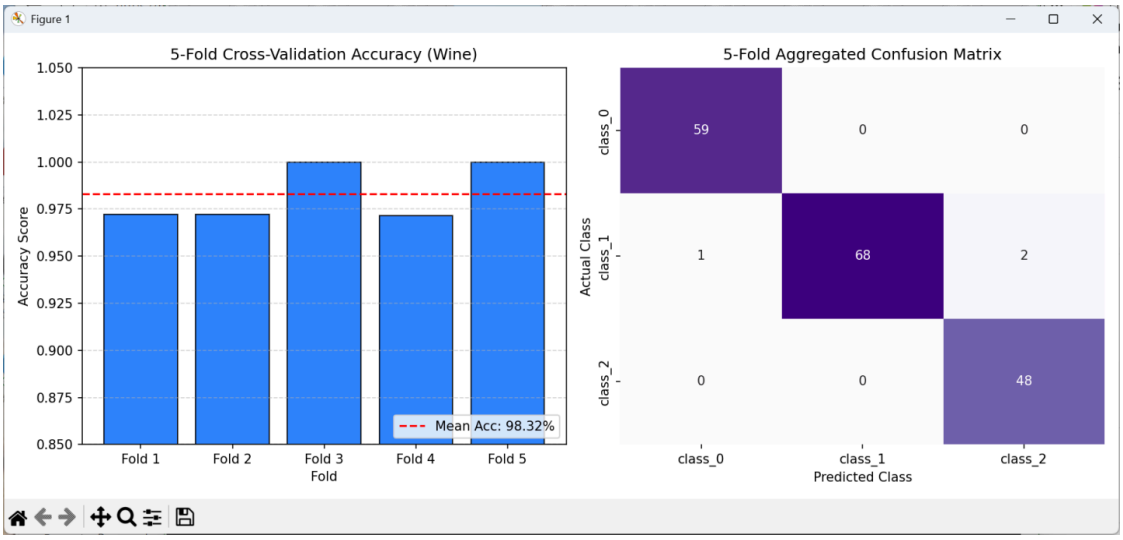
macro avg 0.94 0.94 0.94 150
weighted avg 0.94 0.94 0.94 150
mX5 🐞 executed: 09/10/2026 09:13:33 Runtime: 0:0:6.756 Memload: 70% use RemObjects Pascal Script. Copyright (c) 2004-2026 by RemObjects Software & maXbox5

This maXbox Pascal script tests the **Wine Recognition Dataset** using **5-fold stratified cross-validation** via **TPythonEngine**. It calculates out-of-fold predictions and renders a dual-panel figure with per-fold accuracy scores alongside an aggregated 3×3 **Seaborn confusion matrix heatmap**

How This Script Evaluates the Wine Dataset:

1. **Stratified 5-Fold Splitting (StratifiedKFold)**: Partitions the 178 wine samples into 5 folds while maintaining proportional balance across all three cultivars (Barolo, Grignolino, Barbera).
2. **Out-of-Fold Predictions (cross_val_predict)**: Generates predictions for every sample when it serves in the holdout test fold, preventing data leakage during evaluation.
3. **Dual Visualization**: Renders a side-by-side view showing accuracy stability across folds alongside an annotated Seaborn confusion matrix heatmap.

Advertisement
Privacy Settings



https://sourceforge.net/projects/maxbox5/files/examples/1501_wine_dataset_ml_studio21.pas/download

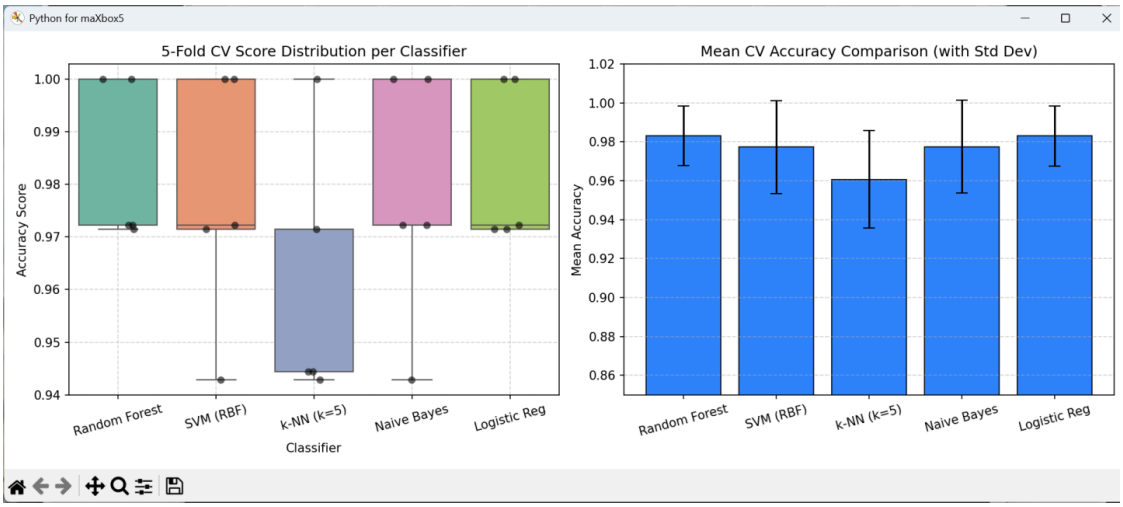
🍷 Would you like to compare cross-validation scores across different classifiers (such as Random Forest vs. SVM vs. k-NN) in maXbox?

This maXbox Pascal script benchmarks **5 core classification algorithms**—Random Forest, Support Vector Machine (RBF kernel), k-Nearest Neighbors, Gaussian Naive Bayes, and Logistic Regression—side-by-side on the **Wine Recognition Dataset** using **5-fold stratified cross-validation**.

📊 Key Visualizations Rendered:

1. **Fold Score Distribution Boxplot (sns.boxplot + sns.stripplot):** Shows the exact spread of accuracy across individual validation folds for each model, highlighting stability vs. variance.
2. **Mean Accuracy Bar Chart (ax.bar with yerr):** Displays the overall mean accuracy per classifier with standard deviation error bars.
3. **Console Benchmark Summary:** Formats a comparison table displaying **`Mean Accuracy`** and **`Std Dev`** directly in the maXbox console.

Advertisement
Privacy Settings



https://sourceforge.net/projects/maxbox5/files/examples/1501_wine_dataset_ml_studio22.pas/download

```
=== MULTI-CLASSIFIER CROSS-VALIDATION BENCHMARK 22 (WINE DATASET)
===
Classifier Mean Accuracy Std Dev
Random Forest 98.32% 1.37%
SVM (RBF) 97.73% 2.13%
k-NN (k=5) 96.06% 2.24%
Naive Bayes 97.75% 2.13%
Logistic Reg 98.30% 1.39%
 mX5 executed: 09/10/2026 11:04:40 Runtime: 0:0:6.528 Memload: 70%
use
```

Some exceptions can occur:

Exception: NotFittedError: This RandomForestClassifier instance is not fitted yet. Call 'fit' with appropriate
arguments before using this estimator. at 852.1654

Exception: SyntaxError: Non-UTF-
8 code starting with '\xb1' on line 1, but no encoding declared;
see https://peps.python.org/pep-
0263/ for details (line 1, offset 31): 'f"{scores.mean()*100.0:.2f}%

(? {scores.std() * 100.0:.2f}%)"' at 853.2857

–> (± {scores.std() } Solution: ({scores.std() }

```
procedure PY_MultiClassifier_CV_Benchmark22;
begin
  with TPythonEngine.Create(Nil) do begin
    try
      loadDLL;
      autofinalize:= false;

      // 1. Import Python ML & Visualization Libraries
      ExecStr('import pandas as pd' + lf +
              'import numpy as np'+lf+
              'import matplotlib.pyplot as plt'+lf+
              'import seaborn as sns');
      execStr('from sklearn.model_selection import StratifiedKFold
              'from sklearn.preprocessing import StandardScaler'+l
              'from sklearn.ensemble import RandomForestClassifier
              'from sklearn.svm import SVC'+lf+
              'from sklearn.neighbors import KNeighborsClassifier'
              'from sklearn.naive_bayes import GaussianNB'+lf+
              'from sklearn.linear_model import LogisticRegression
              'from sklearn import datasets');

      // 2. Load Wine Recognition Dataset
      execstr('import sys,io; output = io.StringIO(); sys.stdout =
      execstr('wine = datasets.load_wine()'+lf+
              'X = wine.data'+lf+
              'y = wine.target');

      // 3. Feature Scaling (StandardScaler)
      execstr('scaler = StandardScaler()'+lf+
              'X_scaled = scaler.fit_transform(X)');

      // 4. Configure Stratified 5-Fold Cross-Validation
      execstr('cv = StratifiedKFold(n_splits=5, shuffle=True, rand

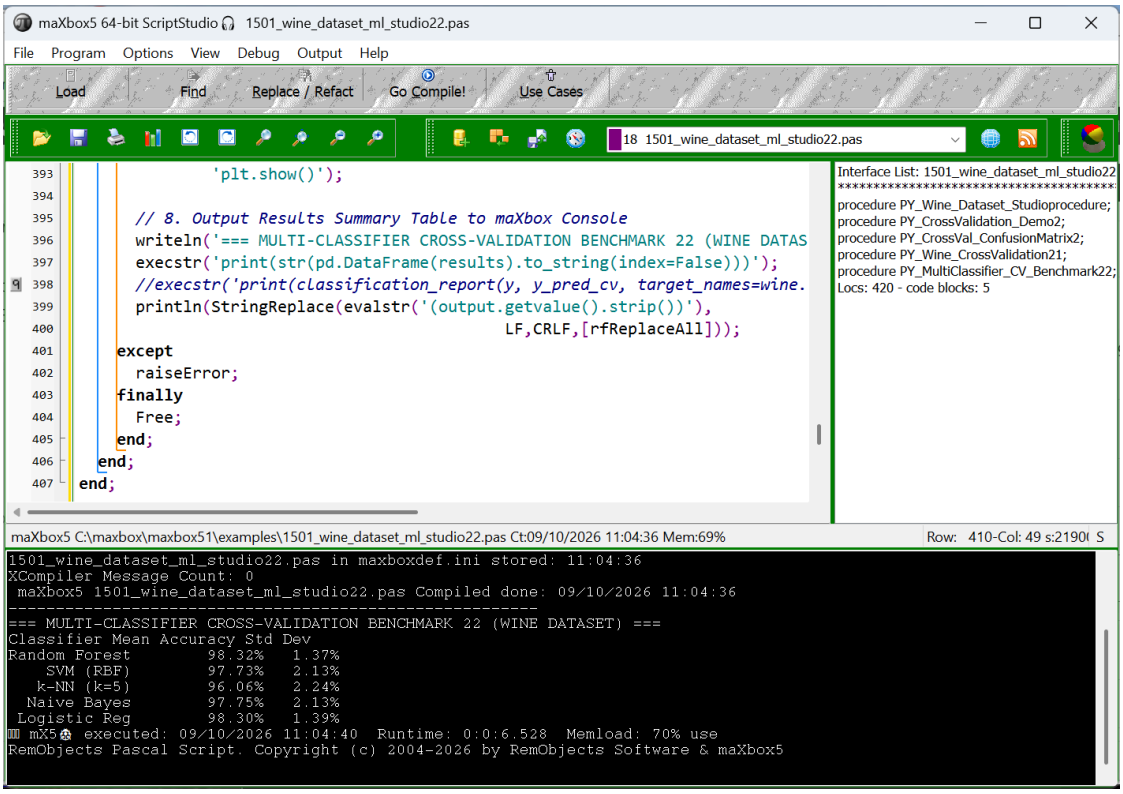
      // 5. Define Classifiers Suite
      execstr('models = {'+lf+
              '    "Random Forest": RandomForestClassifier(n_estim
              '    "SVM (RBF)": SVC(kernel="rbf", C=1.0, random_st
              '    "k-NN (k=5)": KNeighborsClassifier(n_neighbors=
              '    "Naive Bayes": GaussianNB(),'+lf+
              '    "Logistic Reg": LogisticRegression(max_iter=200
              '}');

```

```
      // 6. Evaluate all models across 5 folds
      execstr('results = []'+lf+
              'fold_data = []'+lf+
              'for name, model in models.items():'+lf+
              '    scores = cross_val_score(model, X_scaled, y, cv
              '    results.append({'+lf+
              '        "Classifier": name,'+lf+
              '        "Mean Accuracy": f"{scores.mean()*100:.2f}%
              '        "Std Dev": f"{scores.std()*100:.2f}%"'+lf+
              '    })'+lf+
              '    for fold_idx, score in enumerate(scores):'+lf+
              '        fold_data.append({"Classifier": name, "Fold

      // 7. Render Comparative Boxplot & Bar Chart
      execstr('fold_df = pd.DataFrame(fold_data)'+lf+
              'fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5
              '# Panel 1: Boxplot showing fold score distributions
              'sns.boxplot(data=fold_df, x="Classifier", y="Accura
              'sns.stripplot(data=fold_df, x="Classifier", y="Accu
              'ax1.set_title("5-Fold CV Score Distribution per Cla
              'ax1.set_ylabel("Accuracy Score")'+lf+
              'ax1.tick_params(axis="x", rotation=15)'+lf+
              'ax1.grid(True, linestyle="--", alpha=0.5)'+lf+
              '# Panel 2: Bar Chart of Mean Accuracies with Error
              'summary_df = fold_df.groupby("Classifier", sort=Fal
              'ax2.bar(summary_df["Classifier"],summary_df["mean"]
              'ax2.set_ylim(0.85, 1.02)'+lf+
              'ax2.set_title("Mean CV Accuracy Comparison (with St
              'ax2.set_ylabel("Mean Accuracy")'+lf+
              'ax2.tick_params(axis="x", rotation=15)'+lf+
              'ax2.grid(True, axis="y", linestyle="--", alpha=0.5)
              'plt.tight_layout()'+lf+
              'fig.canvas.manager.set_window_title("Python for maX
              'plt.show()');

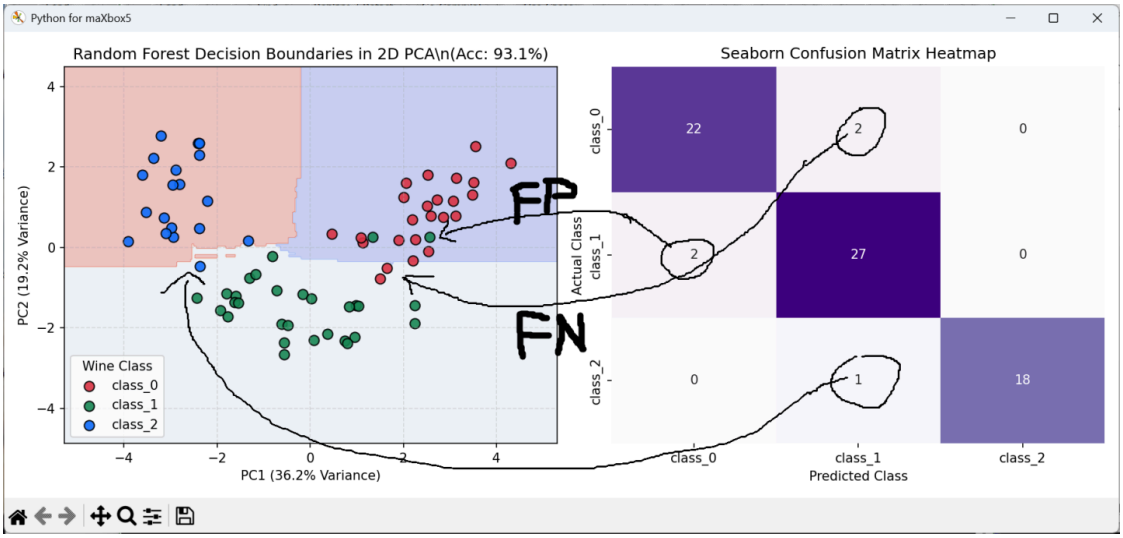
      // 8. Output Results Summary Table to maXbox Console
      writeln('=== MULTI-CLASSIFIER CROSS-VALIDATION BENCHMARK 22
      execstr('print(str(pd.DataFrame(results).to_string(index=Fal
      //execstr('print(classification_report(y, y_pred_cv, target_
      println(StringReplace(evalstr('(output.getvalue().strip())')
                                              LF,CRLF,[rfReplaceAll
    except
      raiseError;
    finally
      Free;
    end;
  end;
end;
```



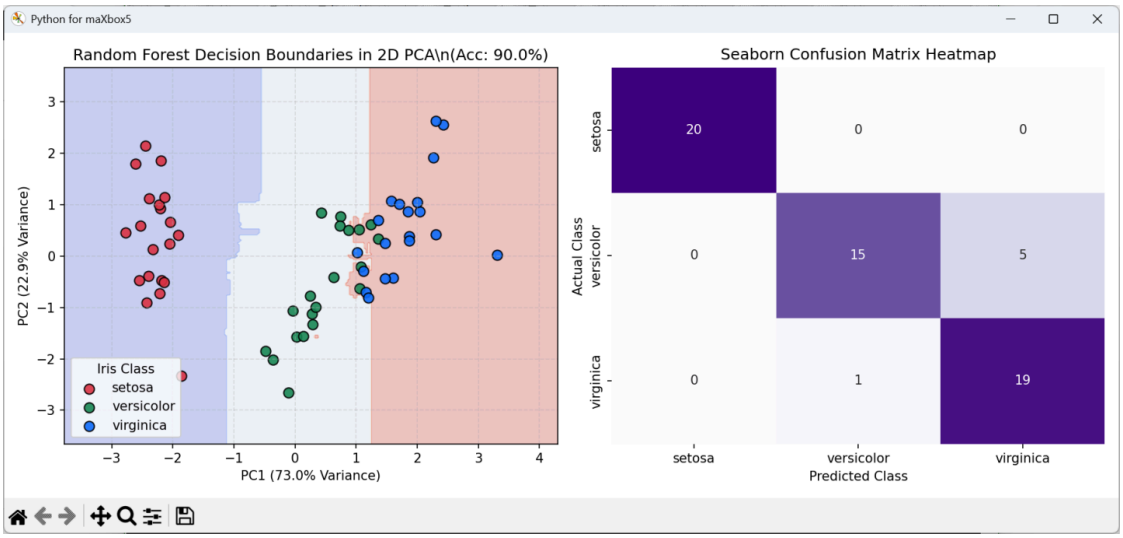
https://sourceforge.net/projects/maxbox5/files/examples/1501_wine_dataset_ml_studio22.pas/download

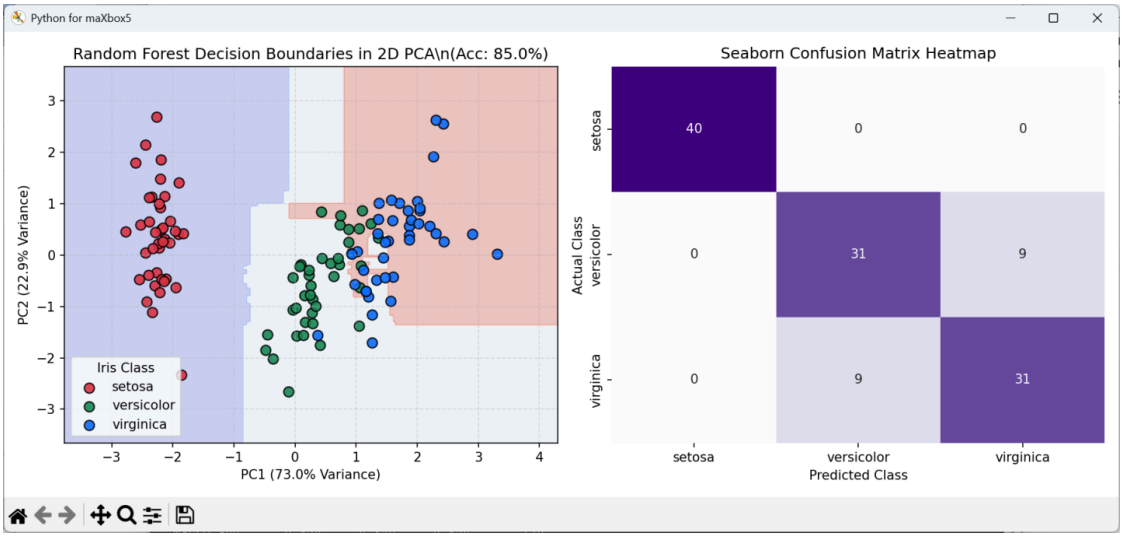
Visualize Missclassification (FP versus FN)

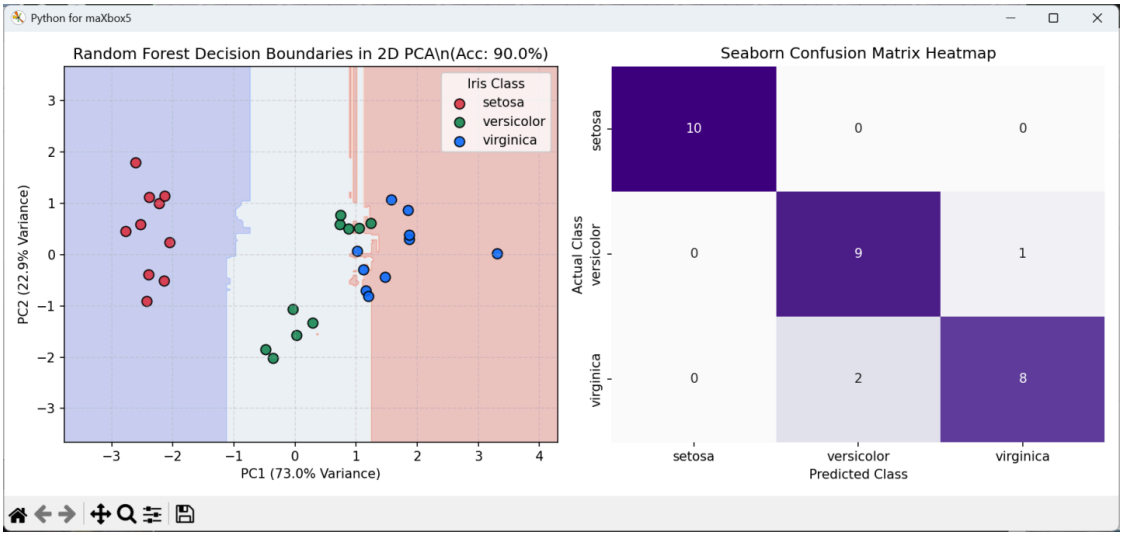
False-Positive Rate (FPR): Answers the question, *"Among truly negative cases, how many were incorrectly flagged?"*



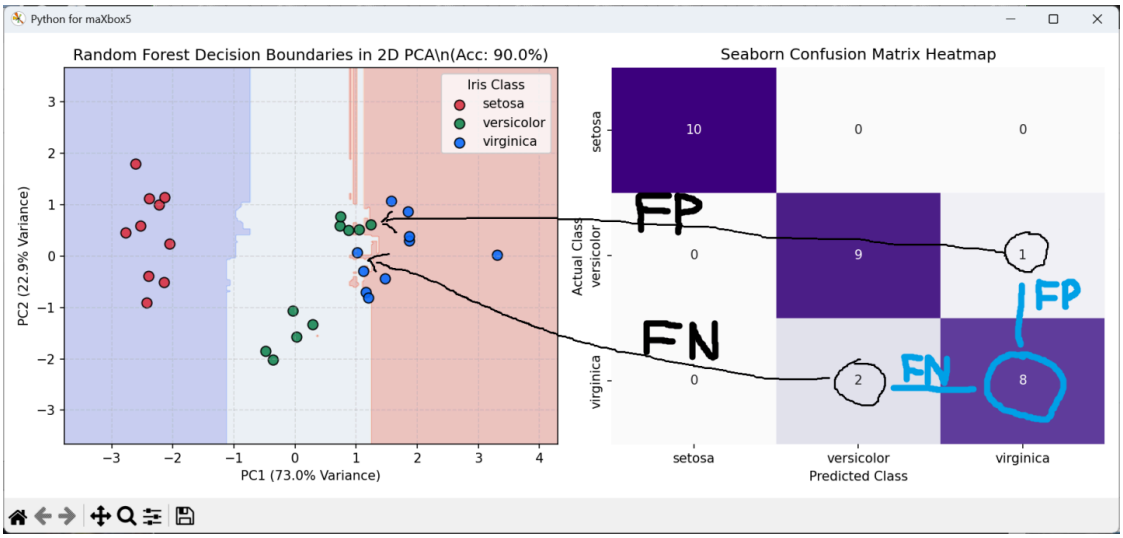
Class 0 has 2 False Positive (the green ones flaged) and has 2 False Negatives (the red ones missed)







https://sourceforge.net/projects/maxbox5/files/examples/1501_wine_dataset_ml_studio23_iriscomp.pas/download



Virginica has 1 False Positive (the green ones flaged) and has 2 False Negatives (the blue ones missed)

Advertisements

Occasionally, some of your visitors may see an advertisement here, as well as a Privacy & Cookies banner at the bottom of the page. You can hide ads completely by upgrading to one of our paid plans.

UPGRADE NOW DISMISS MESSAGE

Posted in Code, Data Science, Machinelearning, maXbox, Python

4 responses to “Machinelearning Steps”



maxkleiner

December 2, 2025 Edit

Python’s future remains bright because its values align with how developers actually learn and build:

- Readability
- Approachability
- Stability
- A touch of irreverence

As AI continues to influence software development—and Octoverse shows that 80% of new developers on GitHub use GitHub Copilot in their first week—Python’s clarity matters

more than ever.

Why developers still flock to Python: Guido van Rossum on readability, AI, and the future of programming – The GitHub Blog

★ Like

Reply



maxkleiner

December 19, 2025 Edit

The whiteboard is blank. The marker is uncapped. The interviewer leans back and says design Twitter. And your brain just explodes into this mess of load balancers and microservices and Kafka streams because you think that's what they want to hear. Two minutes in and you're already lost in your own diagram.

★ Like

Reply



maxkleiner

December 28, 2025 Edit

AI will revolutionise everything. It will be the next mega industry. It will beckon in a new, automated industrial revolution. It will separate growth from labour and generate copious amounts of wealth. This is the narrative that has been shoved down our throats by drug-addled techbros and the pathetic financebros that prop them up. This narrative has been used to leverage so much debt and investment into AI that we have basically bet the entire Western economy on its success. But there is one problem: this narrative simply isn't true, and the AI industry is beginning to find that out the hard way.

★ Like

Reply



Python4Delphi Concepts – Trans Europ Express

September 23, 2026 Edit

[…] It then executes a series of Python code snippets (some from variables like pyCLosure and pyDEco, others built inline) that each demonstrate one of 10 concepts.